

# 目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎

## 2.4 分布式结构化数据表Bigtable

- Bigtable是Google开发的基于GFS和Chubby的分布式**存储**系统
- Google的很多数据，包括Web索引、卫星图像数据等在内的海量**结构化**和**半结构化**数据，都存储在Bigtable中。
- 从实现上看，Bigtable并没有什么全新的技术，但是如何选择合适的技术，并将这些技术高效、巧妙地**结合**在一起，恰恰是最大的难点。
- 本节会对Bigtable的**数据模型**、**系统架构**、**实现**，以及它使用的一些**数据库技术**进行介绍。



## 2.4 分布式结构化数据表Bigtable

- ▶ 2.4.1 设计动机与目标
- 2.4.2 数据模型
- 2.4.3 系统架构
- 2.4.4 主服务器
- 2.4.5 子表服务器
- 2.4.6 性能优化

## 2.4 分布式结构化数据表Bigtable

### • Bigtable 的设计动机

Google运行着目前世界上最繁忙的系统，它每时每刻处理的客户服务请求数量是普通的系统根本无法承受的。

Google目前向公众开放的服务很多，需要处理的数据类型也非常多。包括URL、网页内容、用户的个性化设置在内的数据都是Google需要经常处理的。

需要存储  
的数据种类繁多

海量的  
服务请求

商用数据库  
无法满足需求

一方面，现有商用数据库的设计着眼点在于其通用性，根本无法满足Google苛刻的服务要求，而且在数量庞大的服务器上根本无法部署。另一方面，对于底层系统的完全掌控会给后期的系统维护、升级带来极大的便利。

## 2.4 分布式结构化数据表Bigtable

- Bigtable 应达到的基本目标

广泛的适用性

Bigtable是为了满足一系列Google产品而并非特定产品的存储要求。

很强的可扩展性

根据需要随时可以加入或撤销服务器。

高可用性

确保几乎所有的情况下系统都可用。因为对于用户来说，短暂的服务中断也是不能忍受的。

简单性

底层系统的简单性既可以减少系统出错的概率，也为上层应用的开发带来便利。

## 2.4 分布式结构化数据表Bigtable

2.4.1 设计动机与目标

► 2.4.2 数据模型

2.4.3 系统架构

2.4.4 主服务器

2.4.5 子表服务器

2.4.6 性能优化

## 2.4 分布式结构化数据表Bigtable

### ● 数据模型

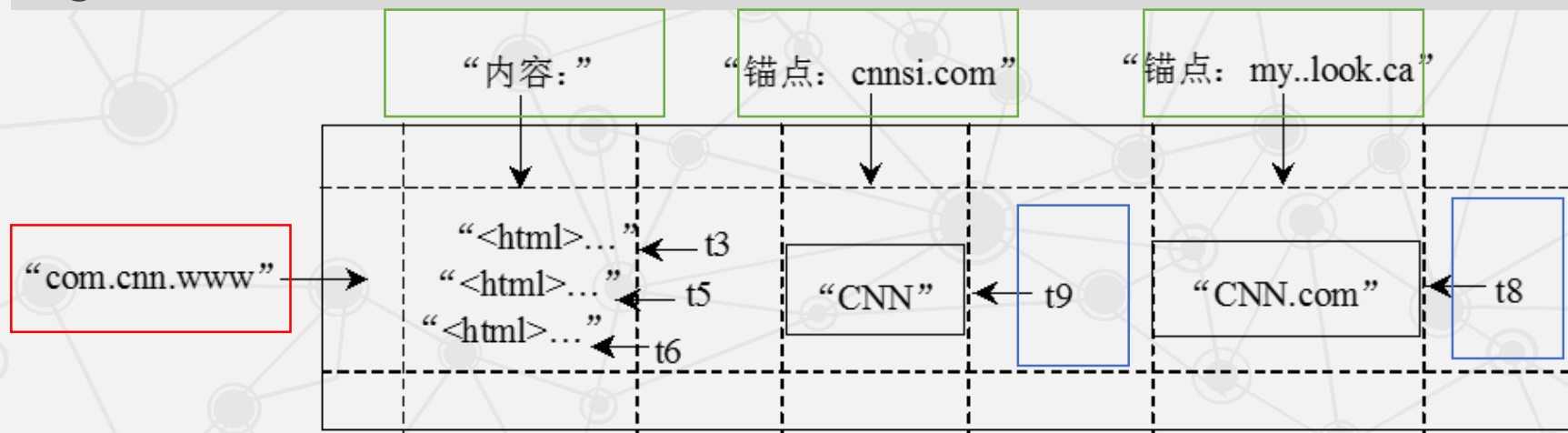
Bigtable是一个分布式多维映射表，表中的数据通过一个**行关键字**（Row Key）、一个**列关键字**（Column Key）以及一个**时间戳**（Time Stamp）进行索引。

Bigtable对存储在其中的数据不做任何解析，一律看做**字符串**，具体数据结构的实现需要用户自行处理。

Bigtable的**存储逻辑**可以表示为：

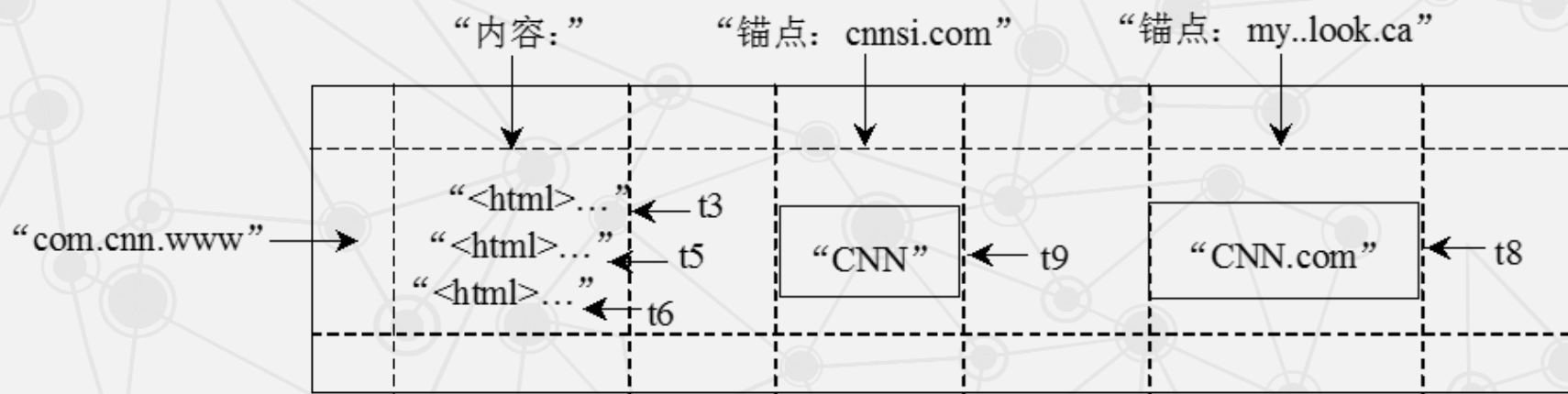
$(\text{row:string, column:string, time:int64}) \rightarrow \text{string}$

Bigtable的**数据存储格式**如下图：





## 2.4 分布式结构化数据表Bigtable

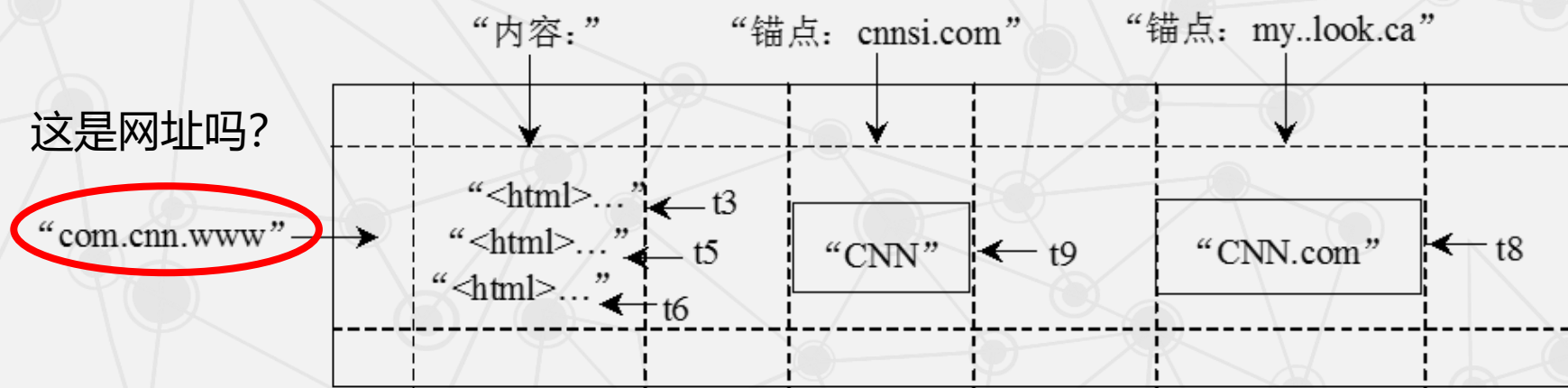


行

- Bigtable的**行关键字**可以是任意的字符串，但是大小不能够超过64KB
- Bigtable和传统的关系型数据库不同，它不支持一般意义上的事务，但能保证对于行的读写操作具有**原子性**（Atomic）。
- 表中数据都是根据**行关键字**进行排序的，排序使用的是**词典序**。
- "com.cnn.www" 就是一个行关键字。



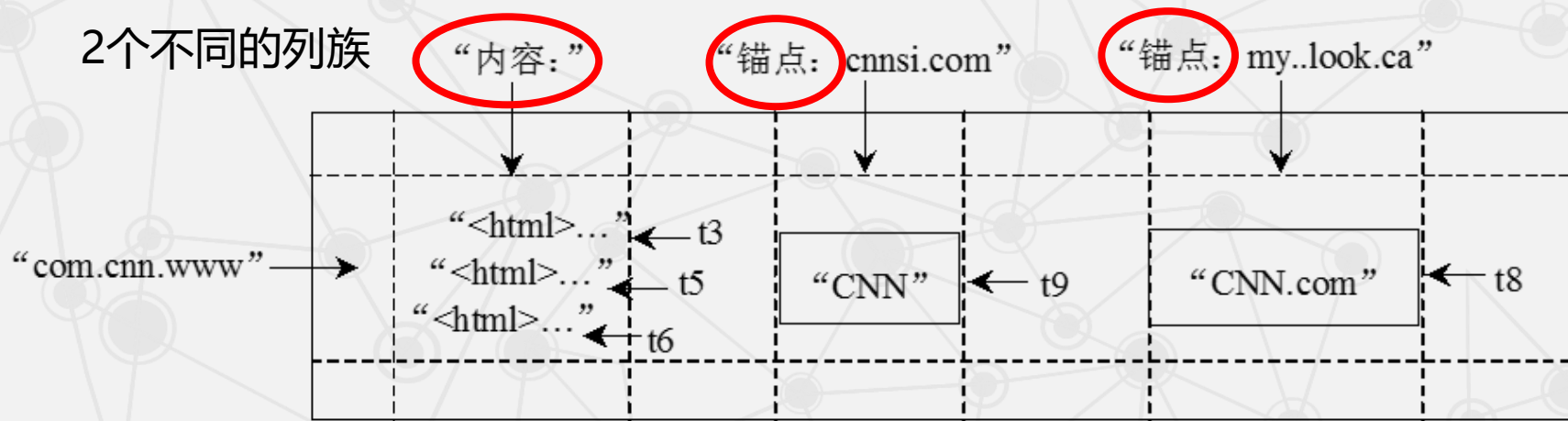
## 2.4 分布式结构化数据表Bigtable



行

- 不直接存储网页地址，而将其**倒排**，是Bigtable的一个巧妙设计。这样做至少会带来以下两个好处：
- 同一**地址域**的网页会被存储在表中的连续位置。方便查找分析
- 倒排便于**数据压缩**，可以大幅提高压缩率。
- 由于规模问题，单个的大表不利于数据的处理，因此，Bigtable将一个表分成了很多**子表**（Tablet），每个子表包含多个行。子表是Bigtable中**数据划分**和**负载均衡**的基本单位。

## 2.4 分布式结构化数据表Bigtable

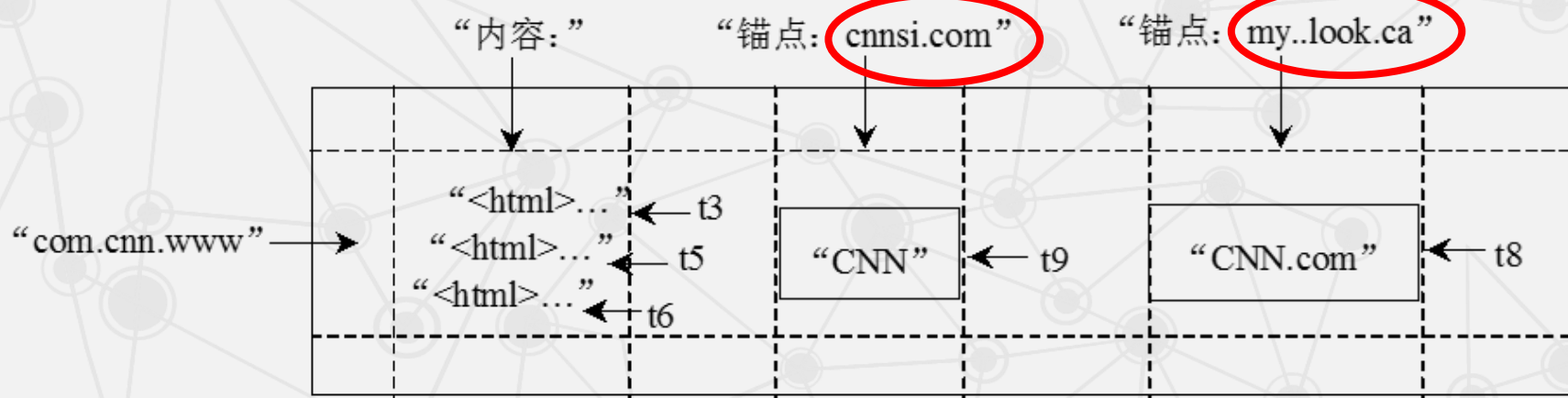


列

- Bigtable并不是简单地存储所有的列关键字，而是将其组织成所谓的**列族** (Column Family)，每个族中的数据都属于同一个类型，并且同族的数据会被**压缩**在一起保存。
- 列关键字的语法规则：**族名: 限定词**  
族名必须有意义，限定词则可以任意选定。

## 2.4 分布式结构化数据表Bigtable

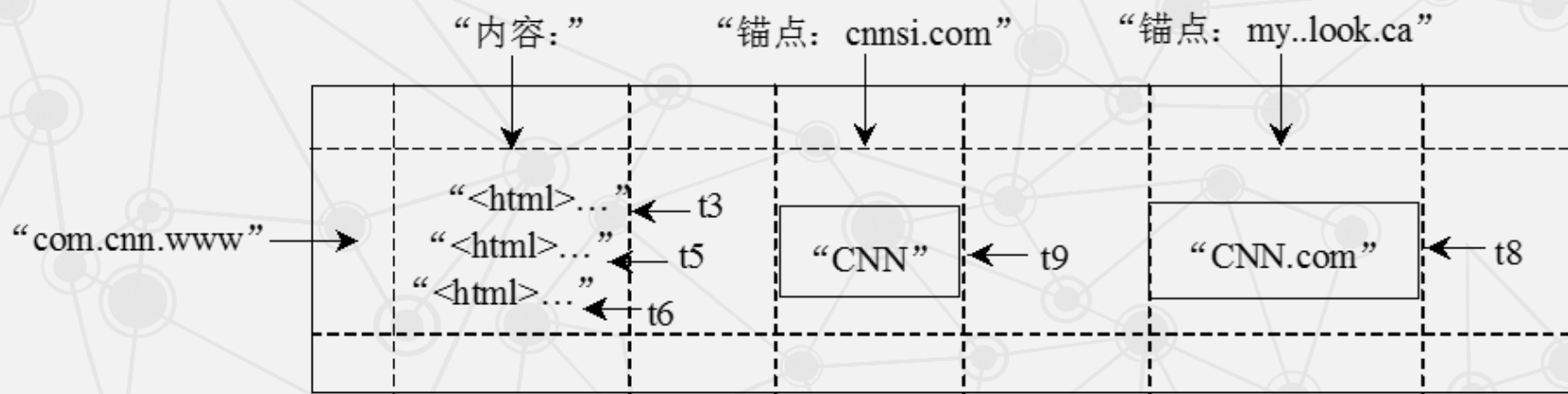
同一个列族中，2个不同的限定词



列

- Bigtable并不是简单地存储所有的列关键字，而是将其组织成所谓的列族 (Column Family)，每个族中的数据都属于同一个类型，并且同族的数据会被压缩在一起保存。
- 列关键字的语法规则：族名：限定词  
族名必须有意义，限定词则可以任意选定。

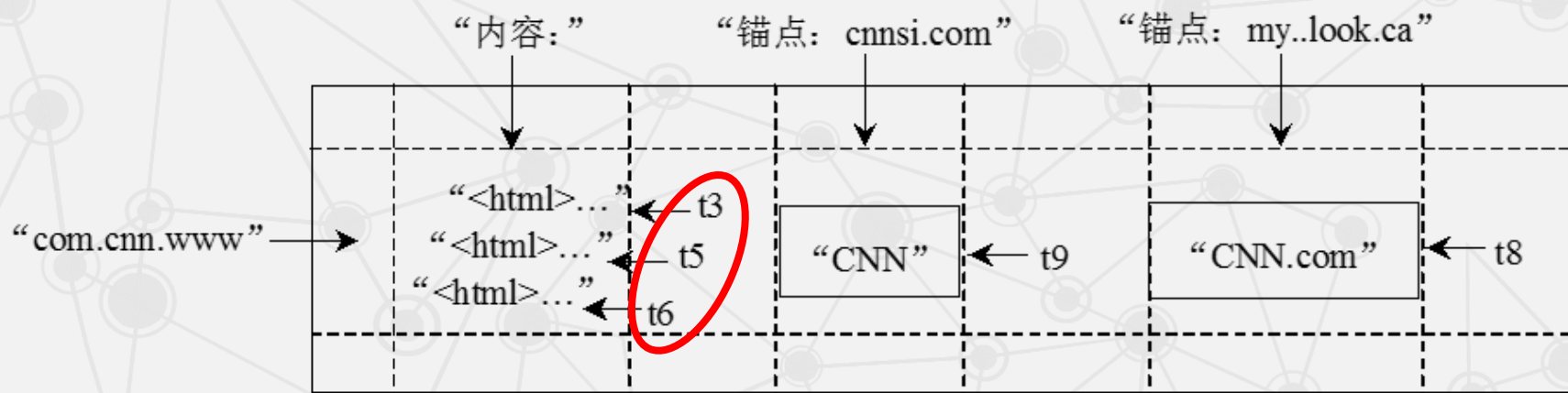
## 2.4 分布式结构化数据表Bigtable



列

- Bigtable并不是简单地存储所有的列关键字，而是将其组织成所谓的**列族**（Column Family），每个族中的数据都属于同一个类型，并且同族的数据会被**压缩**在一起保存。
- 列关键字的语法规则：**族名: 限定词**  
族名必须有意义，限定词则可以任意选定。
- 通过这种方式组织的**数据结构**清晰明了，含义也很清楚。
- 列族同时也是Bigtable中**访问控制**（Access Control）的基本单元。

## 2.4 分布式结构化数据表Bigtable



在t3、t5、t6三个时间获取的网页

时间戳

- Google的很多服务比如网页检索和用户的个性化设置等都需要保存不同时间的数据，这些不同的数据版本必须通过时间戳来区分。
- Bigtable中的时间戳是64位整型数，具体的赋值方式可以由用户自行定义。
- Bigtable允许保留最近的N个版本，或保留限定时间内的所有版本。失效的版本将会由Bigtable的垃圾回收机制自动处理。

## 2.4 分布式结构化数据表Bigtable

(ID photo, photo: cartoon, 1) → ( )

|          | color: blue                                                                                                                                                                                   | color: white                                                                                                                                                                                   | photo: cartoon                                                                                                                                                                                    |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID photo | <div>1</div>                                                                                                 | <div>1</div>  <div>2</div>  | <div>1</div>  <div>2</div>  |
| scene    | <div>1</div>  <div>2</div>  | <div>1</div>                                                                                                 | <div>1</div>  <div>2</div>  |



## 2.4 分布式结构化数据表Bigtable

( ) →



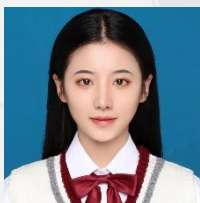
color: blue

color: white

photo: cartoon

ID  
photo

1



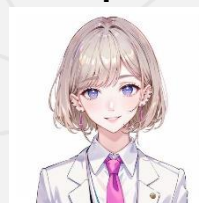
1



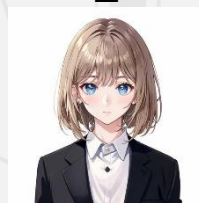
2



1



2



scene

1



2



1



1



2





## 2.4 分布式结构化数据表Bigtable

2.4.1 设计动机与目标

2.4.2 数据模型

▶ 2.4.3 系统架构

2.4.4 主服务器

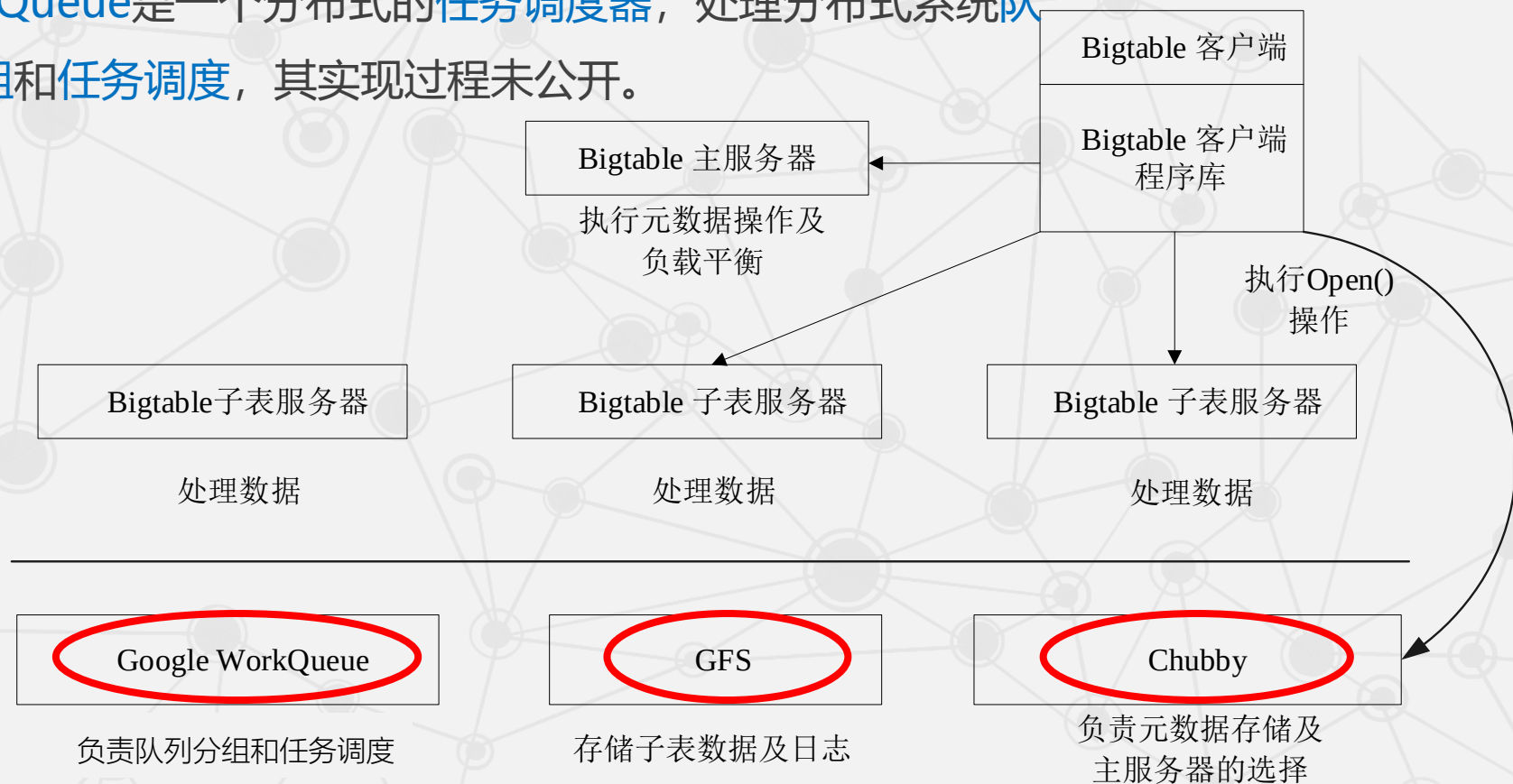
2.4.5 子表服务器

2.4.6 性能优化

## 2.4 分布式结构化数据表Bigtable

### • Bigtable 基本架构

- Bigtable是在Google的另外三个云计算组件基础之上构建的。
- WorkQueue是一个分布式的任务调度器，处理分布式系统队列分组和任务调度，其实现过程未公开。



## 2.4 分布式结构化数据表Bigtable

- Bigtable 中 Chubby 的主要作用

作用一

选取并保证同一时间内只有一个主服务器 (Master Server)。

作用二

获取子表的位位置信息。

作用三

保存Bigtable的模式信息及访问控制列表。

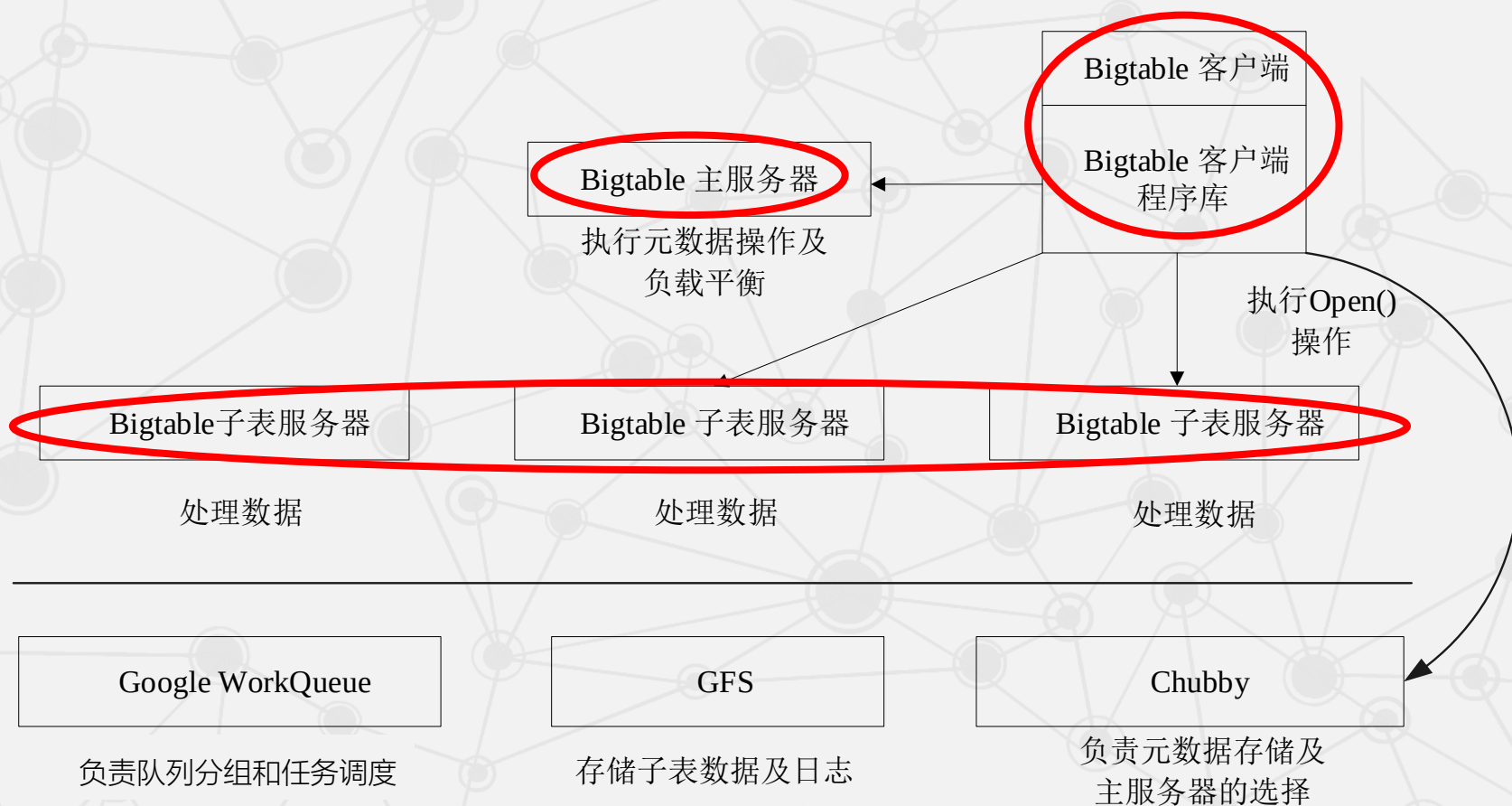
- 另外，在Bigtable的实际执行过程中，Google的MapReduce和Sawzall也被用来改善其性能，但它们并不是Bigtable所必需的。

Sawzall是一种脚本语言，许多语法和表达式与C语言类似。

## 2.4 分布式结构化数据表Bigtable

### • Bigtable 基本架构

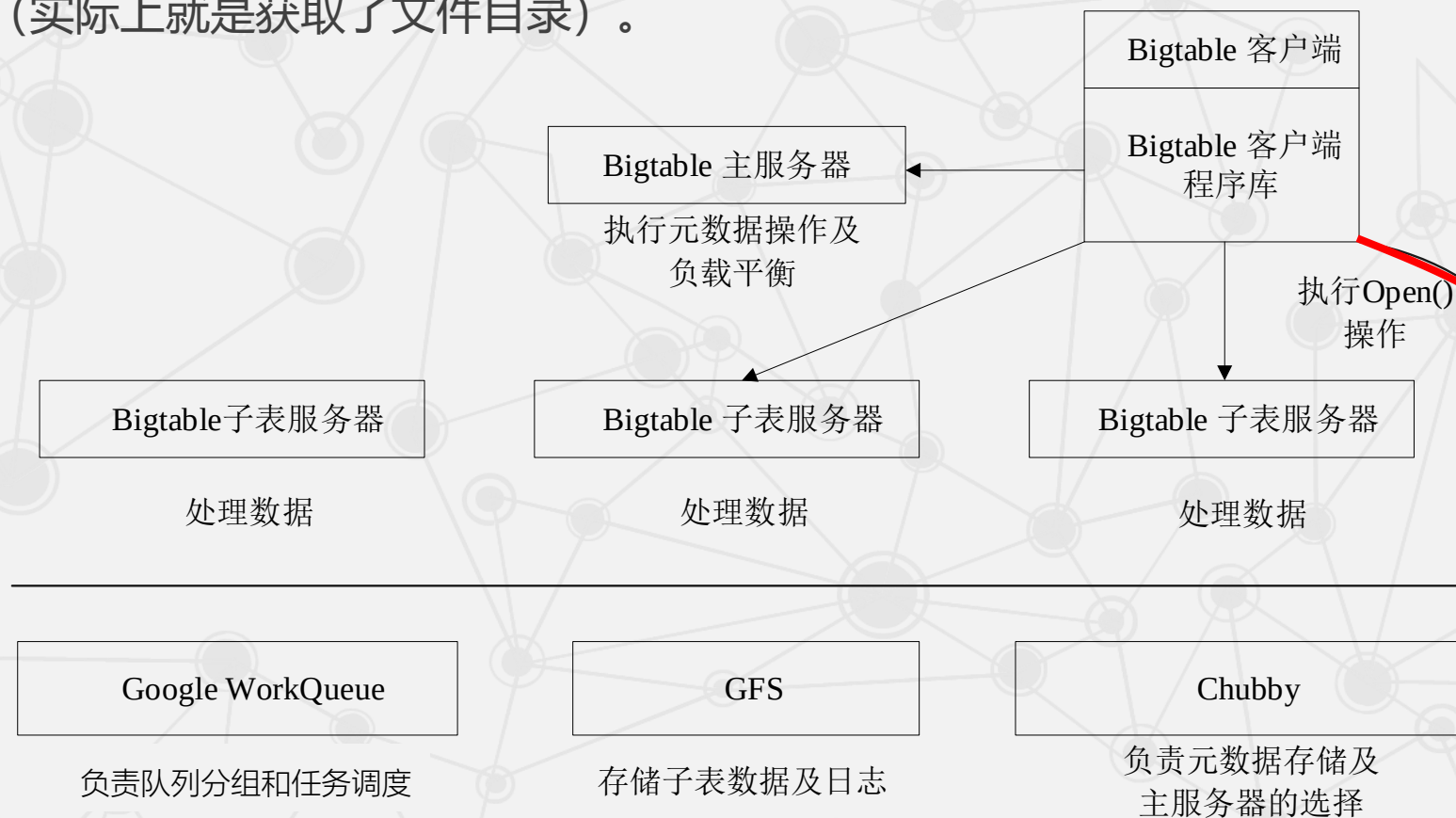
- Bigtable由三部分组成：客户端程序、一个主服务器和多个子表服务器。



## 2.4 分布式结构化数据表Bigtable

### • Bigtable 基本架构

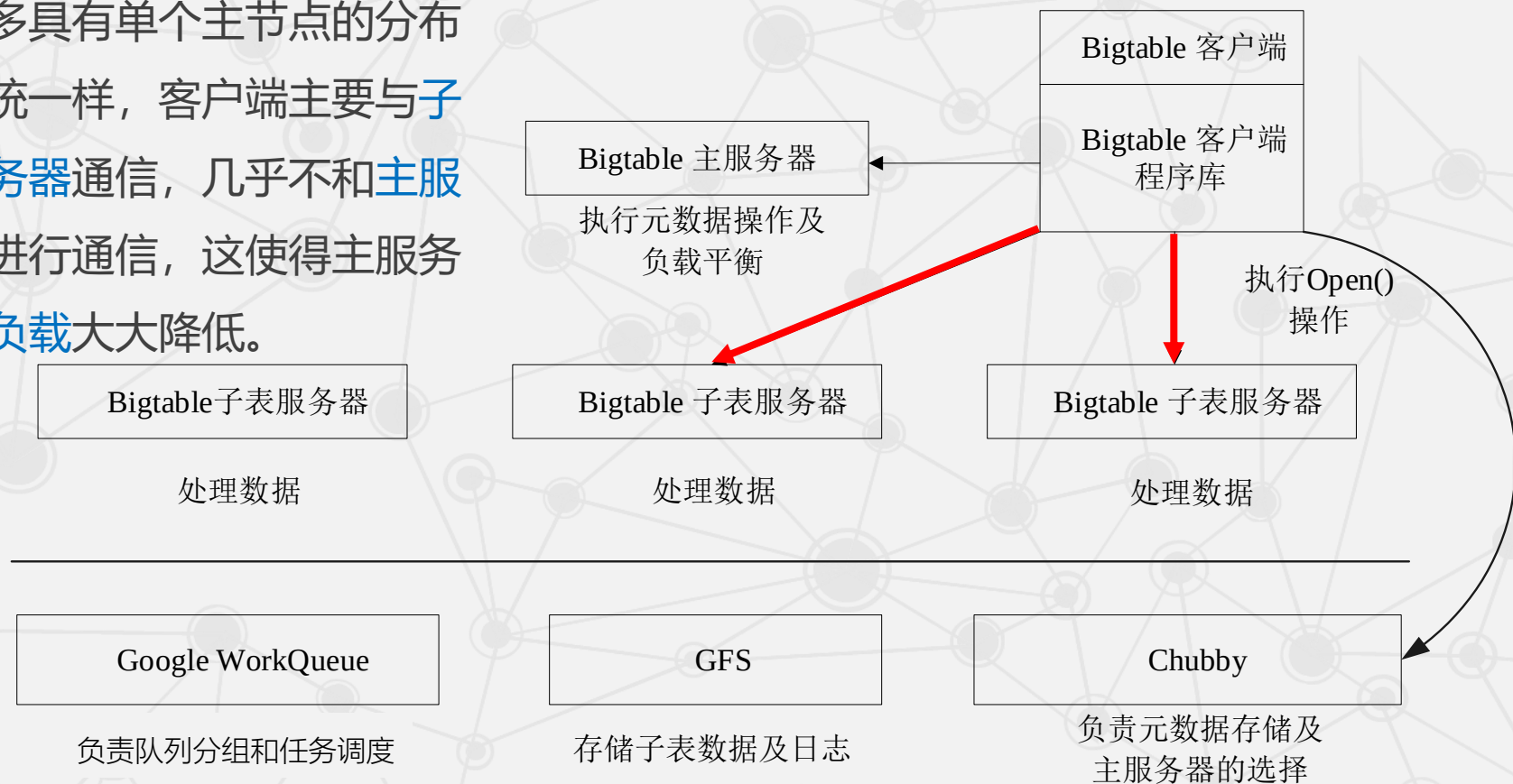
- 客户访问Bigtable服务时，首先要利用其库函数执行open () 操作来打开一个锁（实际上就是获取了文件目录）。



## 2.4 分布式结构化数据表Bigtable

### • Bigtable 基本架构

- 锁打开以后，客户端就可以和子表服务器进行通信了。
- 和许多具有单个主节点的分布式系统一样，客户端主要与子表服务器通信，几乎不和主服务器进行通信，这使得主服务器的负载大大降低。





## 2.4 分布式结构化数据表Bigtable

### • Bigtable 基本架构

- 主服务器主要进行一些元数据的操作，以及子表服务器之间的负载调度问题，实际的数据是存储在子表服务器上的。



## 2.4 分布式结构化数据表Bigtable

2.4.1 设计动机与目标

2.4.2 数据模型

2.4.3 系统架构

► 2.4.4 主服务器

2.4.5 子表服务器

2.4.6 性能优化

## 2.4 分布式结构化数据表Bigtable

主服务器的主要作用：



## 2.4 分布式结构化数据表Bigtable

主服务器的主要作用：

新子表  
分配

主服务器

当一个新的子表产生时，主服务器通过一个**加载**命令将其分配给一个空间足够的子表服务器。

**创建**新表、表**合并**以及较大子表的**分裂**都会产生一个或多个新子表。

**创建**新表和表**合并**，主服务器会自动检测到，因为它们是由**主服务器发起**的。

较大子表的**分裂**由**子表服务器发起**，主服务器无法自动检测到，因此，分割完成之后，子服务器需要向主服务发出一个通知。

## 2.4 分布式结构化数据表Bigtable

主服务器的主要作用：



主服务器必须对子表服务器的状态进行**监控**，以便及时检测到服务器的加入或撤销。

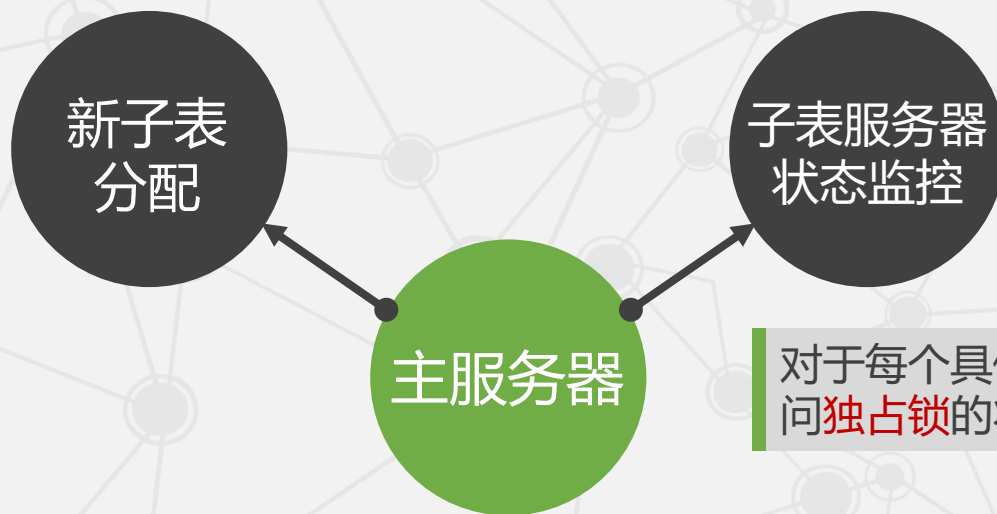
Bigtable主服务器对子表服务器的监控是由**Chubby**完成，子表服务器在初始化时都会得到**独占锁**。

通过这种方式，所有的子表服务器基本信息被保存在Chubby中一个称为**服务器目录**的特殊目录中。

**主服务器**通过检测这个目录来获取最新的子表服务器信息，包括目前活跃的子表服务器，以及每个子表服务器上现已分配的子表。

## 2.4 分布式结构化数据表Bigtable

主服务器的主要作用：



对于每个具体的子表服务器，主服务器会**定期**向其询问**独占锁**的状态。

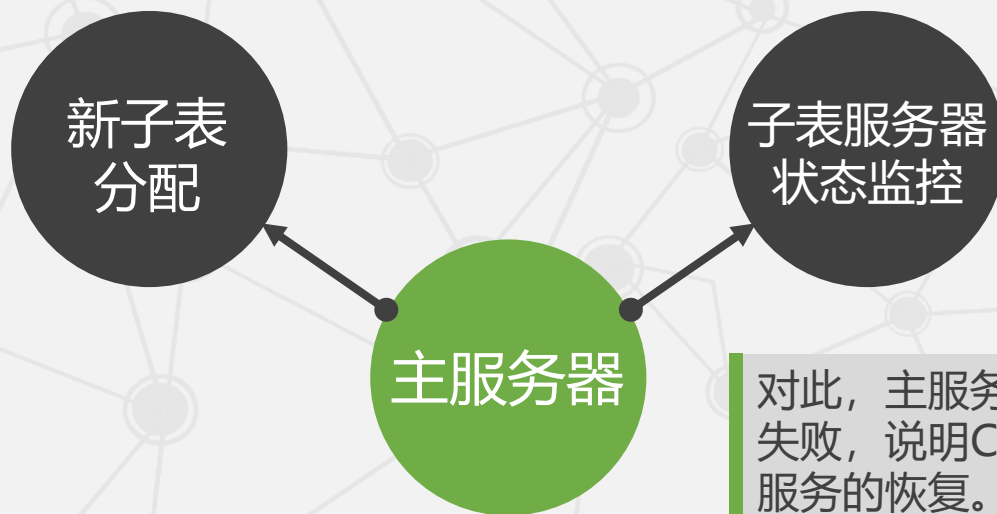
如果子表服务器的锁丢失，或没有回应，则此时可能出现**两种情况**：

要么是Chubby出现了问题（虽然这种情况概率很小，但的确存在。），要么是子表服务器自身出现了问题。



## 2.4 分布式结构化数据表Bigtable

主服务器的主要作用：



对此，主服务器首先自己尝试获取这个独占锁，如果失败，说明Chubby服务出现问题，须等待Chubby服务的恢复。

如果成功，则说明Chubby服务良好而子表服务器本身出现了问题。

这种情况下，主服务器会中止这个子表服务器，并将其上的子表全部移交至其他子表服务器。

## 2.4 分布式结构化数据表Bigtable

主服务器的主要作用：



当在状态监测时，发现某个子表服务器上负载过重时，主服务器会自动对其进行**负载均衡**操作。

## 2.4 分布式结构化数据表Bigtable

主服务器的主要作用：

主服务器

基于系统出现故障是一种常态的设计理念，每个主服务器都被设定了一个会话时间的限制。

当某个主服务器到时退出后，管理系统就会指定一个新的主服务器。

这个新的主服务器的启动需要四个步骤。

## 2.4 分布式结构化数据表Bigtable

### 主服务器的启动需要4个步骤：

从Chubby中获取一个独占锁，确保同一时间只有一个主服务器

步骤 1

步骤 2

扫描服务器目录，发现目前活跃的子表服务器。

与所有的活跃子表服务器取得联系，以便了解所有子表的分配情况

步骤 3

步骤 4

完成这四个步骤后，主服务器就可以正常运行了！

通过扫描元数据表，发现未分配的子表并将其分配到合适的子表服务器。

- 如果元数据表未分配，则首先需要将未分配的子表加入根子表中。
- 由于根子表保存了其他所有元数据子表的信息，确保了扫描能够发现所有未分配的子表。

## 2.4 分布式结构化数据表Bigtable

2.4.1 设计动机与目标

2.4.2 数据模型

2.4.3 系统架构

2.4.4 主服务器

► 2.4.5 子表服务器

2.4.6 性能优化

## 2.4 分布式结构化数据表Bigtable

### • 1. SSTable 及子表基本结构

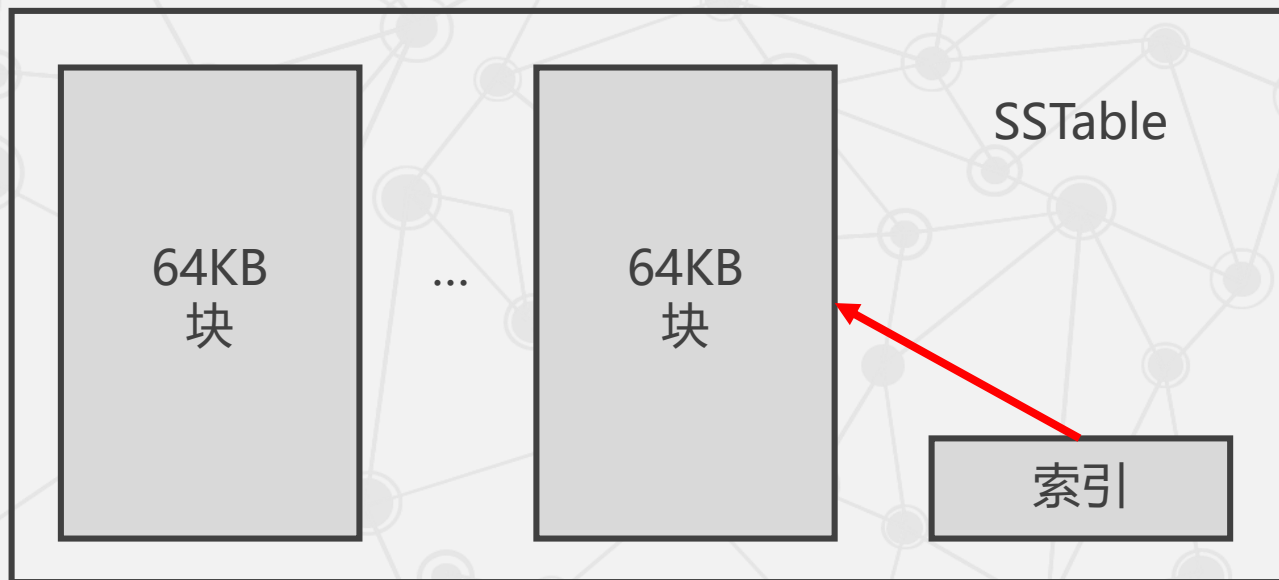
**SSTable**是Google为Bigtable设计的**内部数据存储格式**。所有的SSTable文件都存储在GFS上，用户可以通过**键**来查询相应的值。



SSTable中的数据被划分成一个个**块** (Block)，每个块的大小**可变**，通常为64KB。SSTable的结尾有一个**索引** (Index)，其中保存了块的**位置**。

## 2.4 分布式结构化数据表Bigtable

- 1. SSTable 及子表基本结构



SSTable打开时，索引会被[加载到内存](#)。这样用户在查找某个块时，首先在内存中查找块的[位置](#)信息，然后在硬盘上直接找到这个块，这种查找方法速度非常快。



## 2.4 分布式结构化数据表Bigtable

- 1. SSTable 及子表基本结构

由于每个SSTable一般都不是很大，用户还可以选择将其整体加载进内存，这样查找起来会更快。



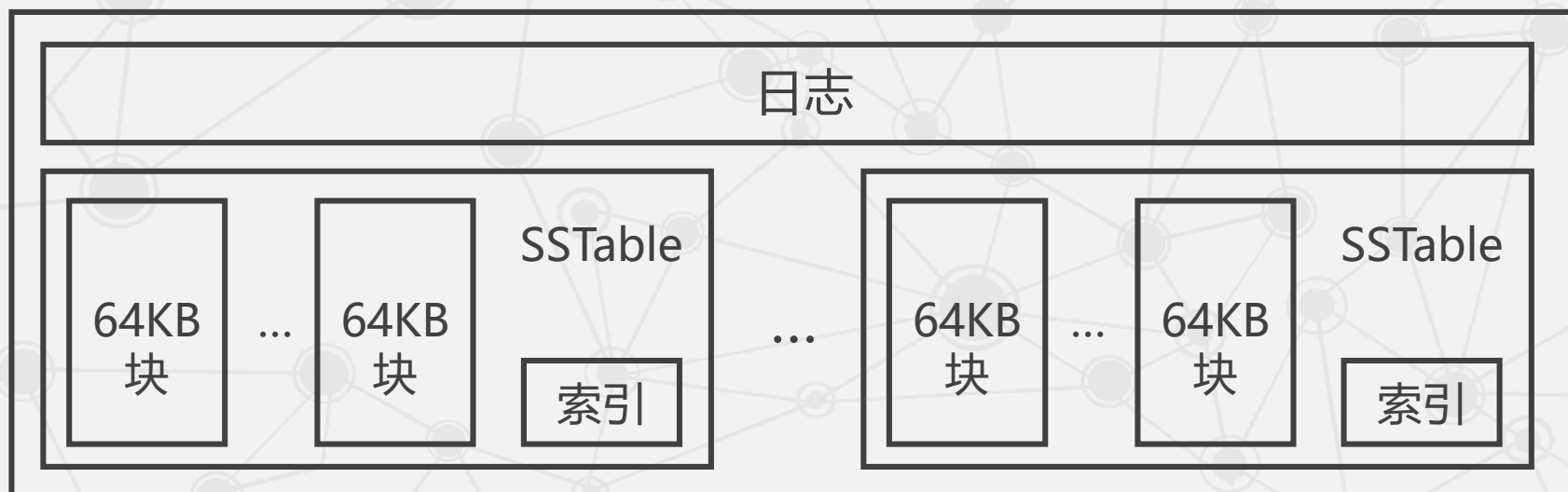
## 2.4 分布式结构化数据表Bigtable

### • 1. SSTable 及子表基本结构

从概念上讲，子表是表中一系列行的集合，它在系统中的实际组成如下图。（子表是Bigtable中数据划分和负载均衡的基本单位。）

每个子表都由多个SSTable以及日志（Log）文件构成。但不同子表的SSTable可以共享，即SSTable会参与多个子表的构成。

Bigtable中的日志文件是一种共享日志，即每个子表服务器上仅保存一个日志文件，某个子表日志只是这个共享日志的一个片段。

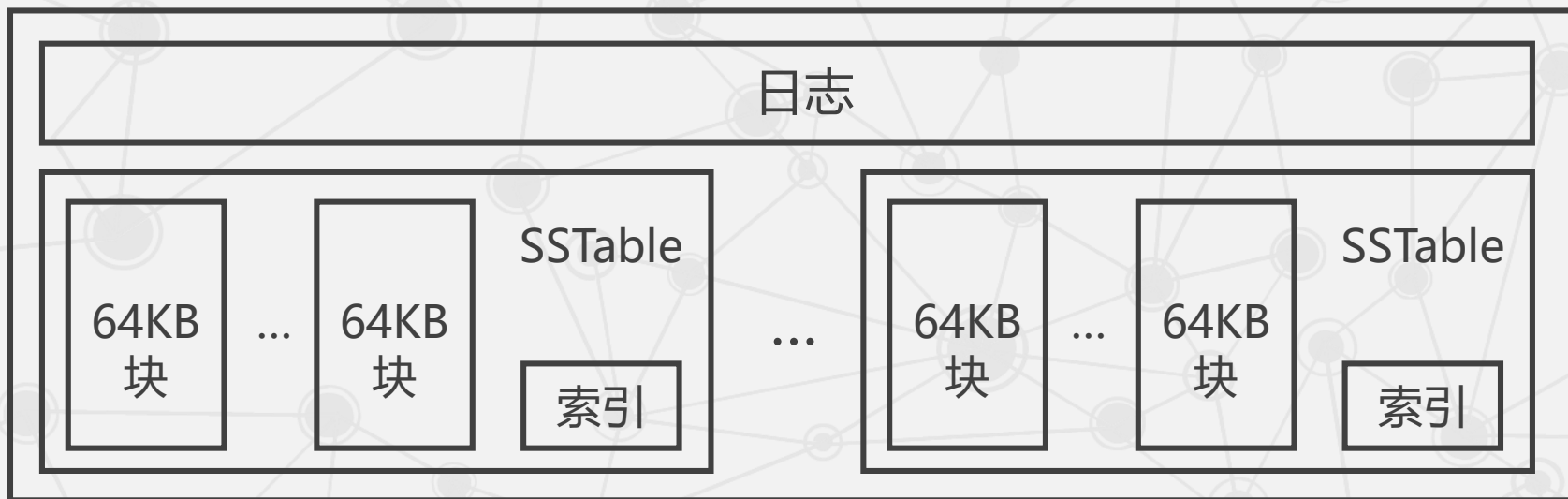


## 2.4 分布式结构化数据表Bigtable

### • 1. SSTable 及子表基本结构

由于不同的子表可能会被分配到不同的子表服务器上，所以，在恢复时每个子表服务器都需要读取[整个共享日志](#)来获取其对应的子表日志。

为了避免此种情况，Bigtable将日志的内容[按照键值排序](#)，这样，不同的子表服务器都可以[连续读取](#)日志文件了。

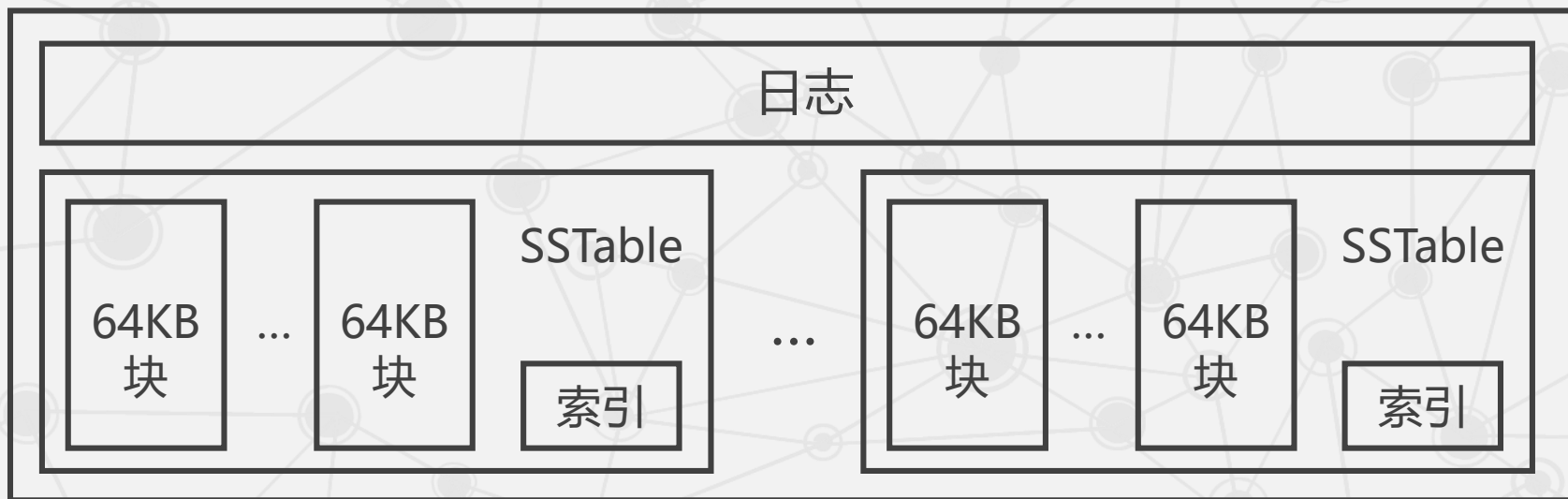


## 2.4 分布式结构化数据表Bigtable

### • 1. SSTable 及子表基本结构

一般每个子表的大小在100MB到200MB之间。

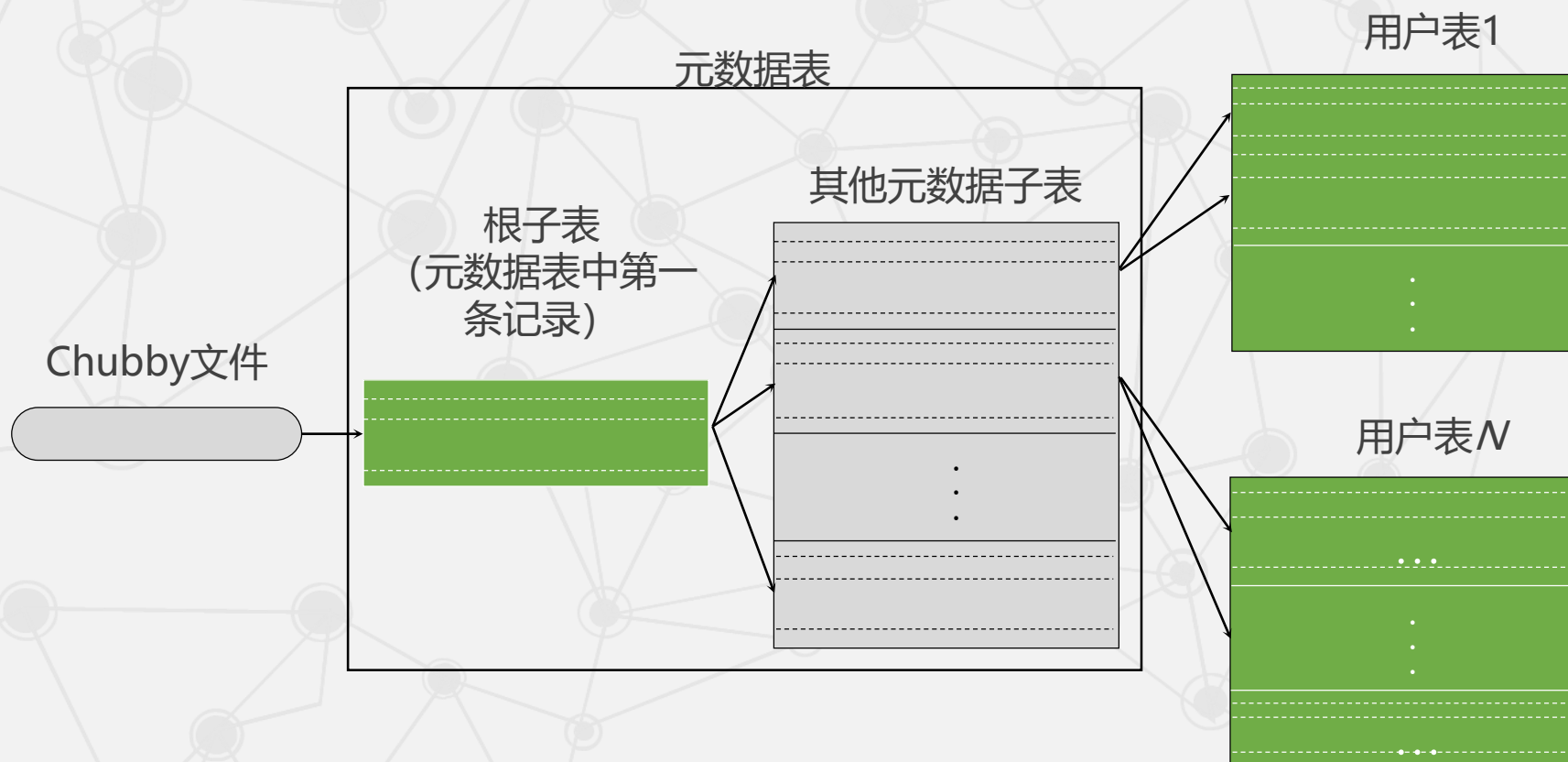
每个子表服务器上保存的子表数量可以从几十到上千不等，通常情况下是100左右。



## 2.4 分布式结构化数据表Bigtable

### ● 2. 子表地址

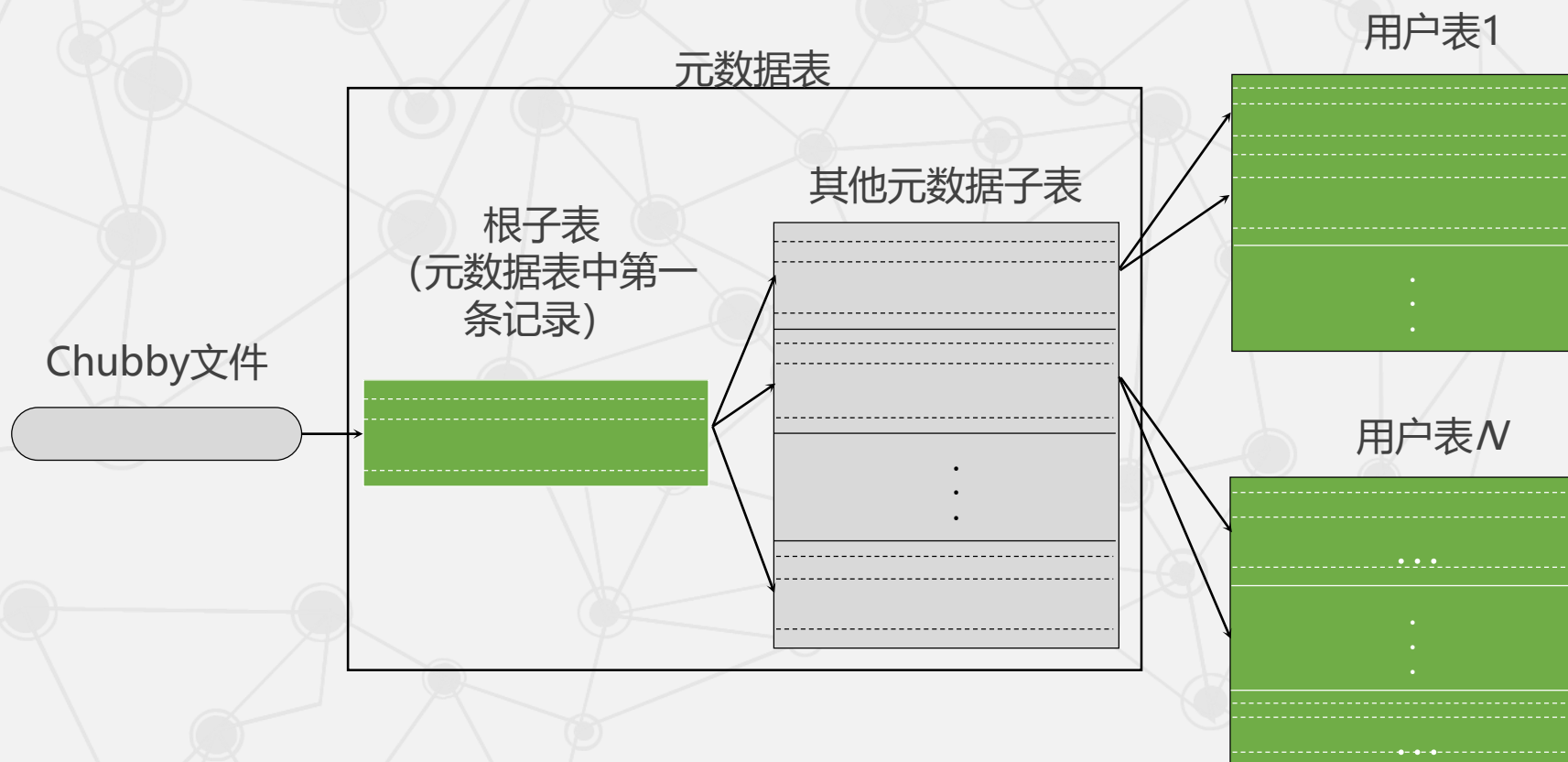
Bigtable系统的内部对子表地址的查询采用的是一种类似B+树的三层查询体系。



## 2.4 分布式结构化数据表Bigtable

### ● 2. 子表地址

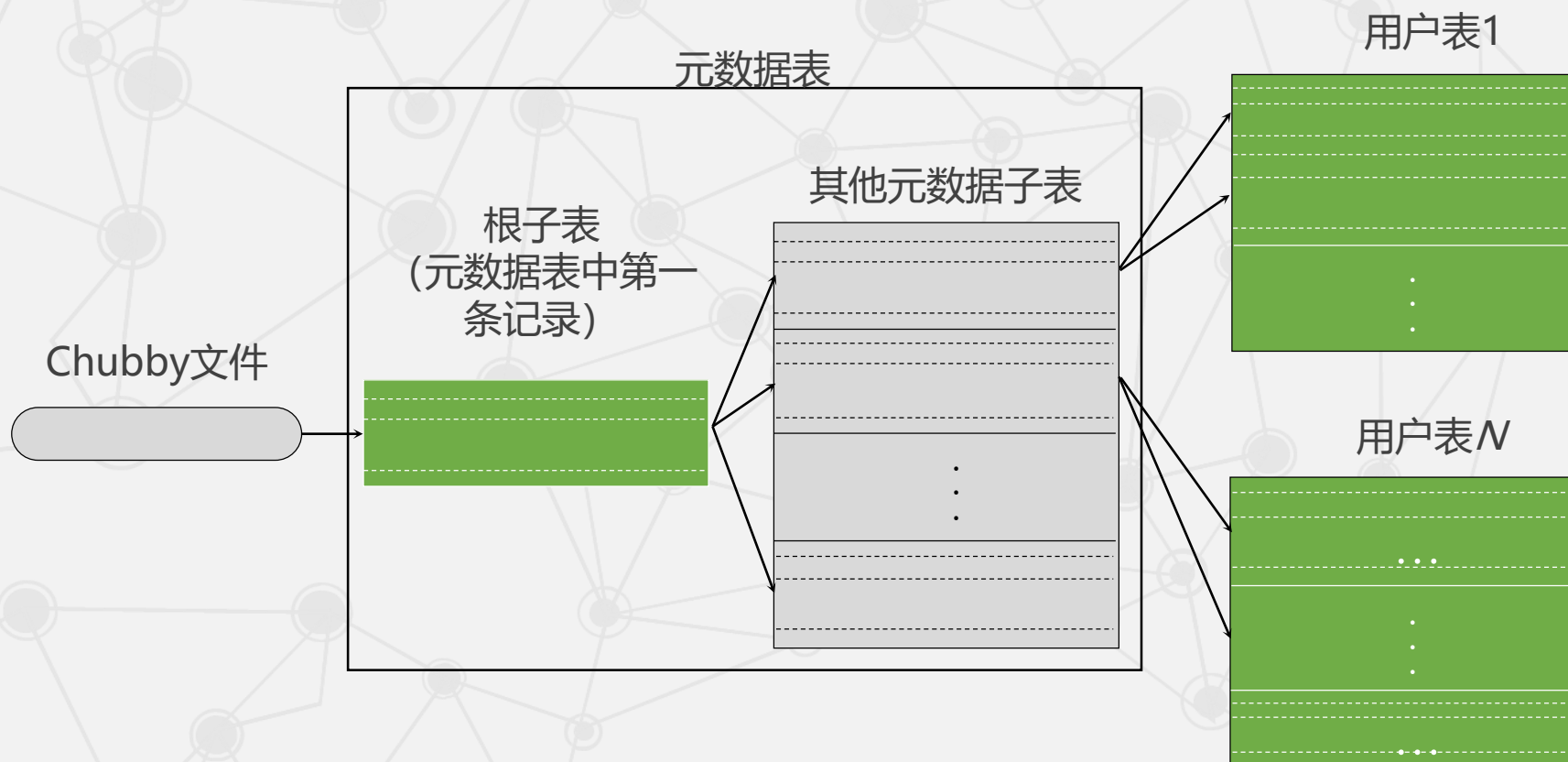
所有的子表地址都被记录在元数据表中，元数据表也是由一个个的元数据子表 (Metadata Tablet) 组成。



## 2.4 分布式结构化数据表Bigtable

### ● 2. 子表地址

**根子表**是元数据表中一个比较特殊的子表，它既是元数据表的第一条记录，也包含了其他元数据子表的地址，同时Chubby中的一个文件也存储了这个根子表的信息。



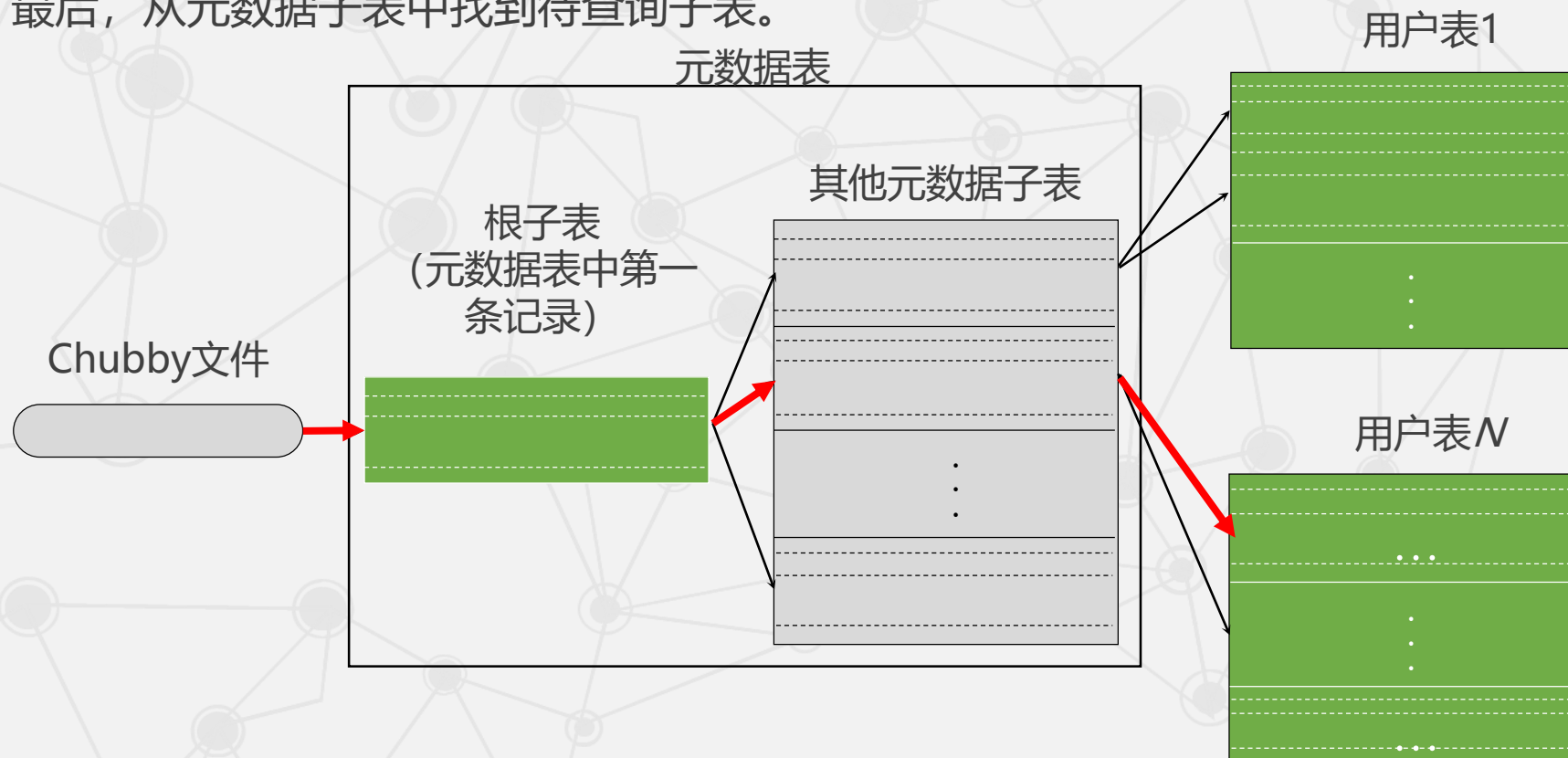


## 2.4 分布式结构化数据表Bigtable

### ● 2. 子表地址

查询时，  
首先，从Chubby中提取根子表的地址；  
进而，读取所需的元数据子表的位置；  
最后，从元数据子表中找到待查询子表。

除了这些子表的元数据之外，元数据表中还保存了其他一些有利于调试和分析的信息，比如事件日志等。



## 2.4 分布式结构化数据表Bigtable

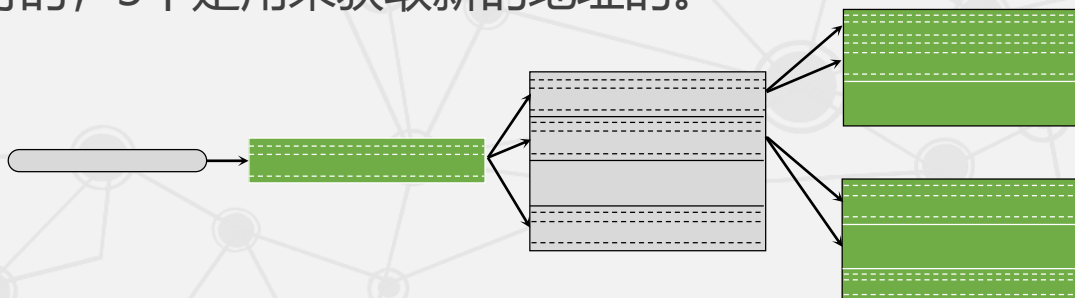
### ● 2. 子表地址

为了减少开销，提高客户访问效率，Bigtable使用了缓存（Cache）和预存取（Prefetch）技术。

#### 缓存

子表的地址信息被缓存在客户端，客户寻址时直接根据缓存信息进行查找。

- 一旦出现缓存为空或缓存信息过时的情况，客户端就要按照下图的方式重新寻址。
- 在缓存为空时，需要3个来回的通信进行寻址；
- 当缓存的信息是过时的，则需要6个来回的通信进行寻址。其中，3个是用来确定过时的，3个是用来获取新的地址的。



## 2.4 分布式结构化数据表Bigtable

### ● 2. 子表地址

为了减少开销，提高客户访问效率，Bigtable使用了缓存（Cache）和预存取（Prefetch）技术。

#### 缓存

子表的地址信息被缓存在客户端，客户寻址时直接根据缓存信息进行查找。

#### 预存取

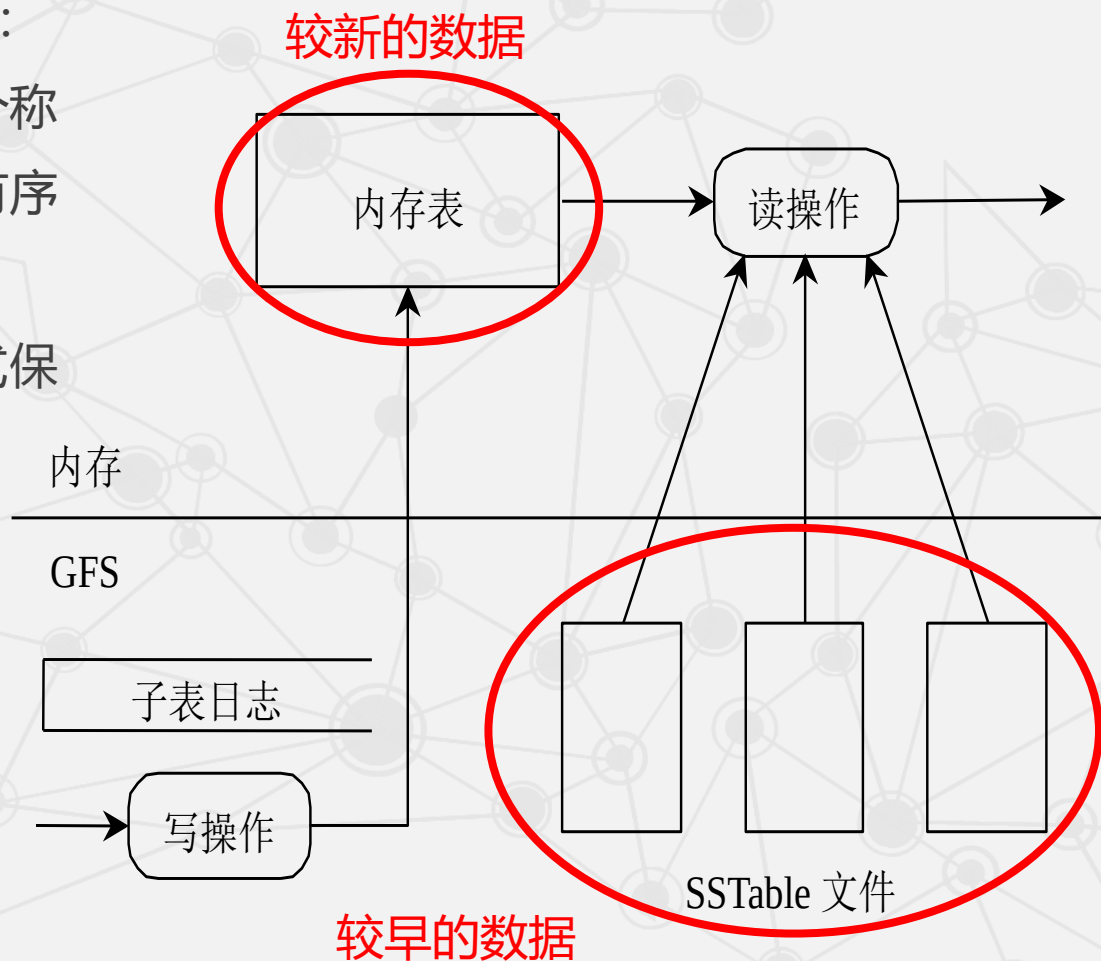
在每次访问元数据表时，不仅仅读取所需的子表元数据，而是额外读取多个子表的元数据，这样下次需要时，就不用再次访问元数据表了。

## 2.4 分布式结构化数据表Bigtable

### ● 3. 子表数据存储及读/写操作

Bigtable将数据存储划分成**两块**:

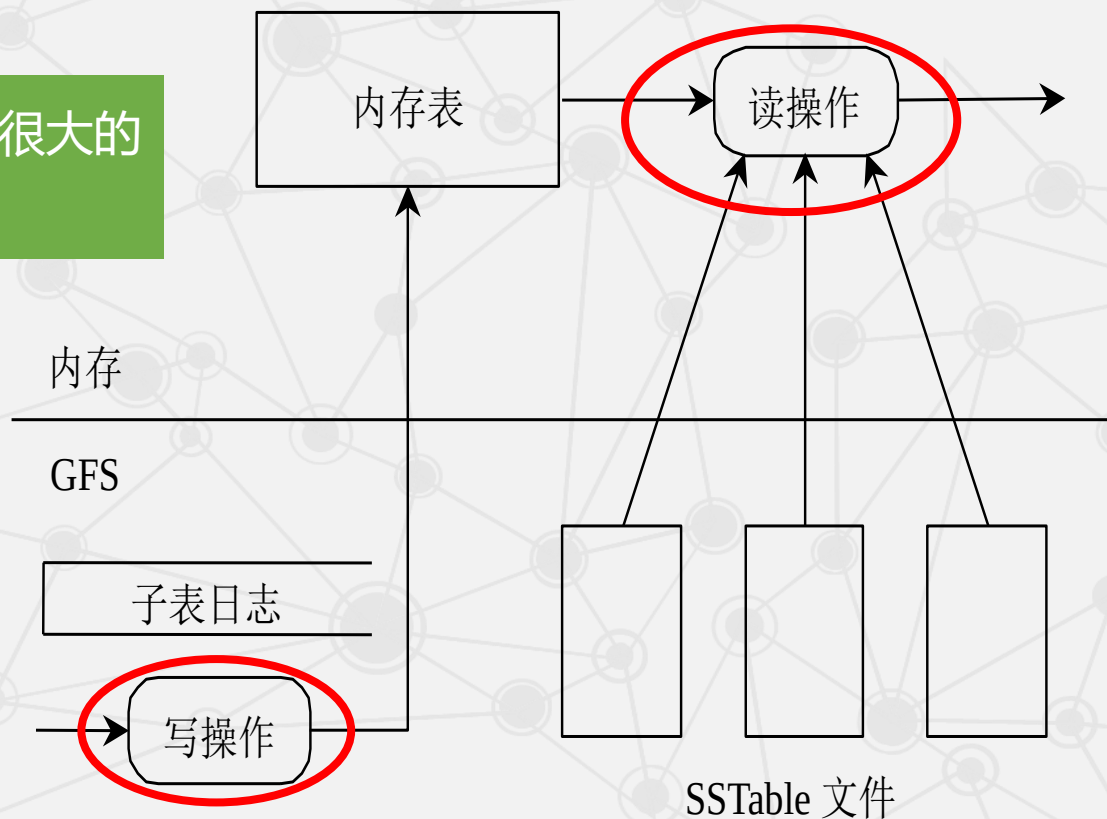
- 较新的数据存储在内存中一个称为**内存表** (Memtable) 的有序**缓冲**里
- 较早的数据则以**SSTable**格式保存在**GFS**中。



## 2.4 分布式结构化数据表Bigtable

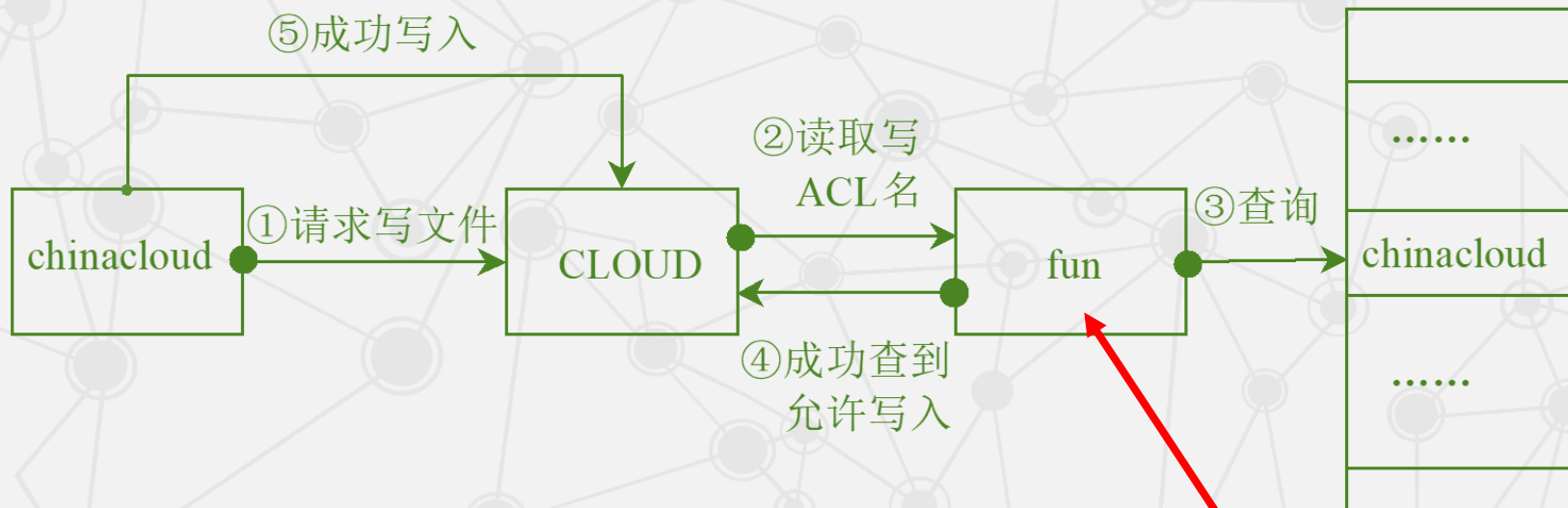
### • 3. 子表数据存储及读/写操作

从图中可以看出，读和写操作有很大的差异性。



## 2.4 分布式结构化数据表Bigtable

### • 3. 子表数据存储及读/写操作

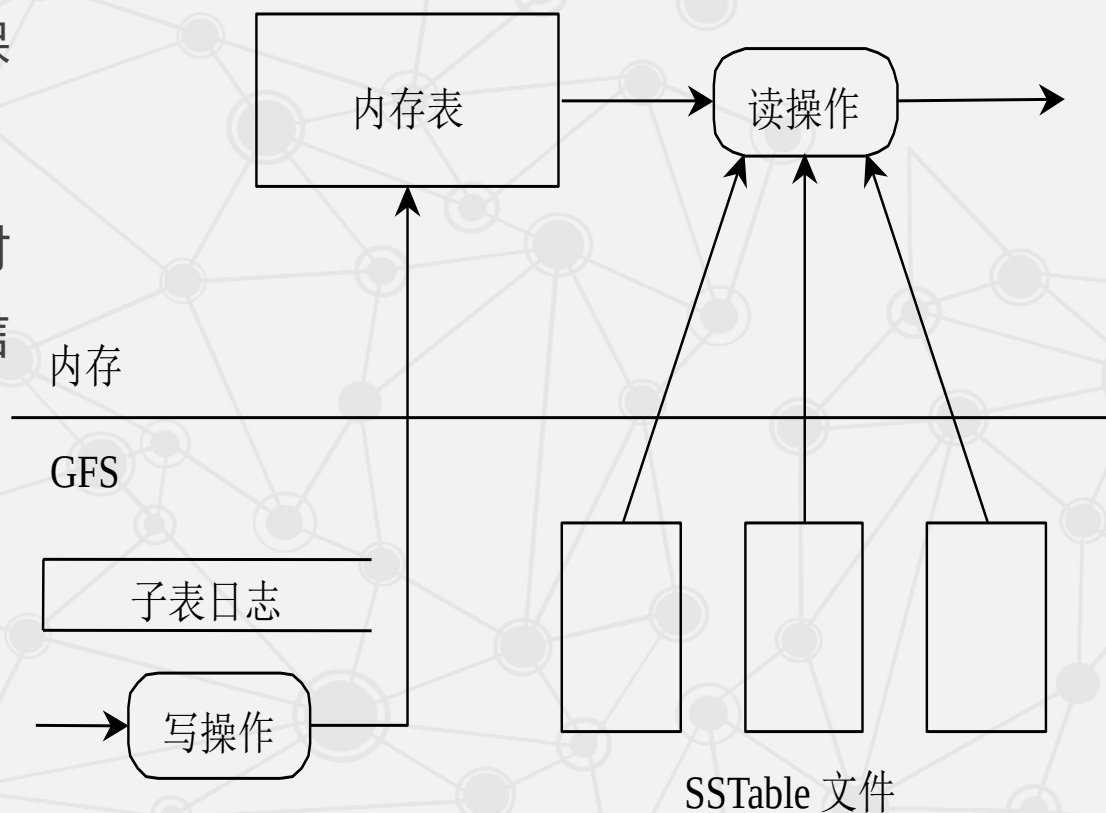


- 做**写操作** (Write Op) 时, 首先查询**Chubby**中保存的**访问控制列表**, 确定用户具有相应的写权限,
- 通过认证后, 写入的数据首先被保存在提交的**日志** (Commit Log) 中。

## 2.4 分布式结构化数据表Bigtable

### • 3. 子表数据存储及读/写操作

- 提交日志中以**重做**记录的形式保存着最近的一系列数据更改，
- 这些重做记录在子表进行**恢复**时可以向系统提供已完成的更改信息。
- 数据提交成功之后，就被写入

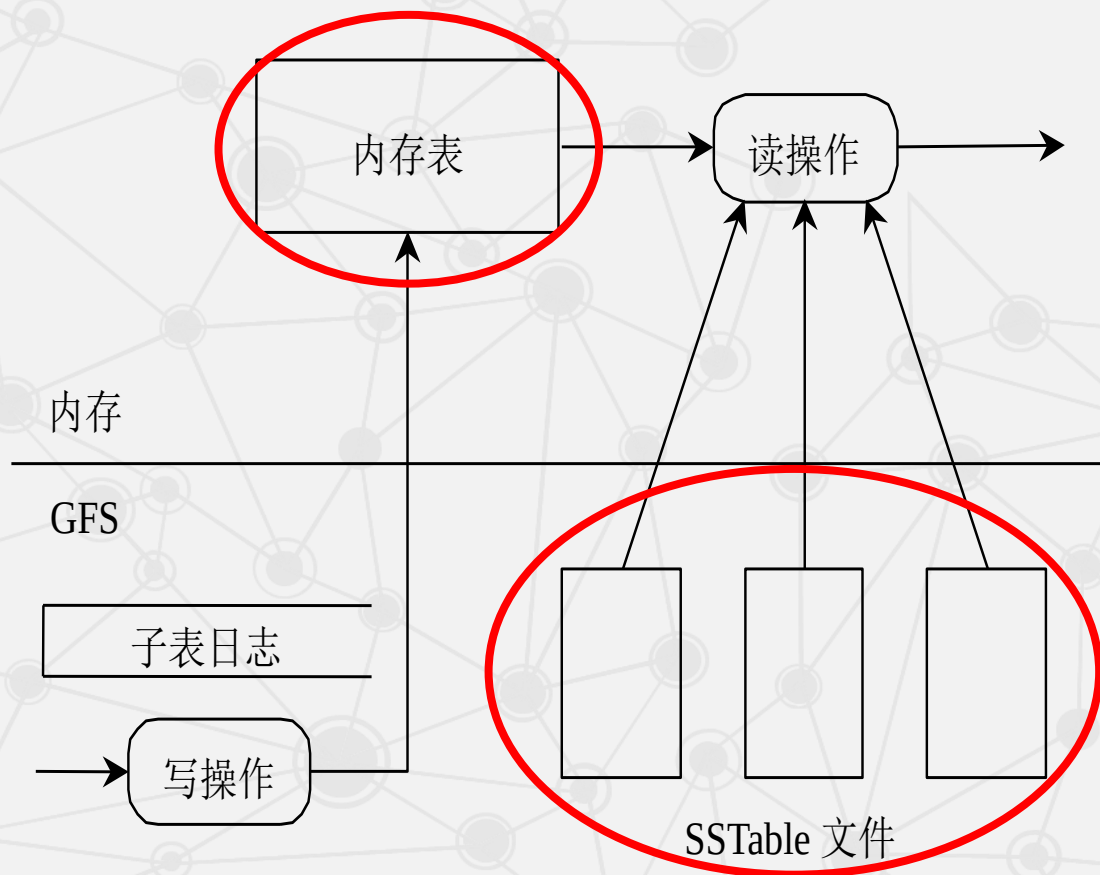




## 2.4 分布式结构化数据表Bigtable

### • 3. 子表数据存储及读/写操作

- 在做读操作时，首先还是要通过认证（Chubby ACL），
- 之后读操作就要结合内存和SSTable文件来进行，因为内存表和SSTable中都保存了数据。



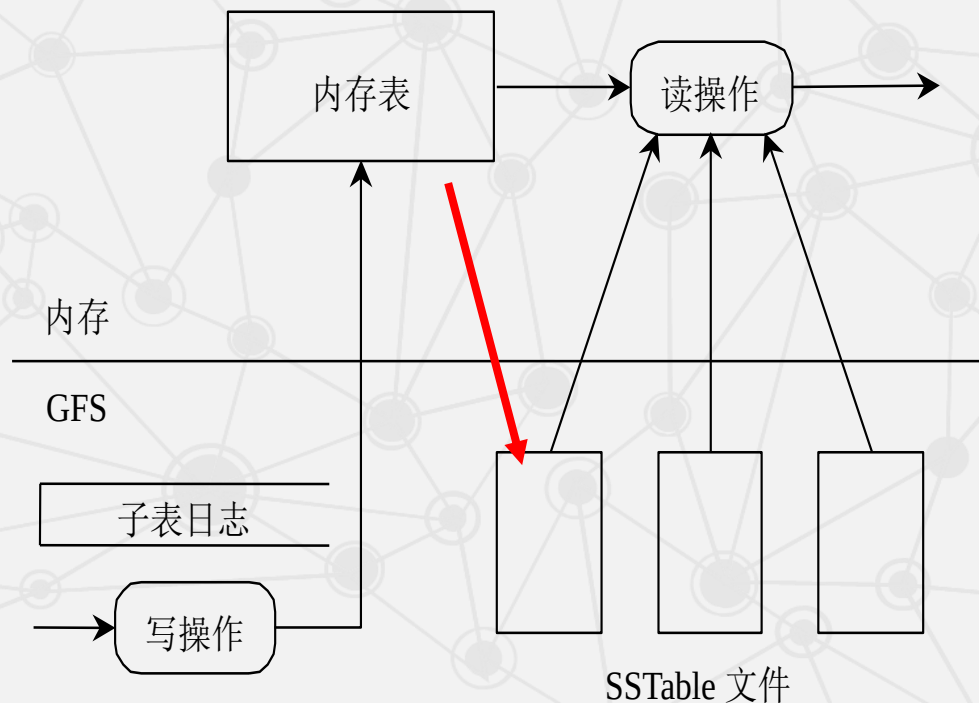
## 2.4 分布式结构化数据表Bigtable

### ● 3. 子表数据存储及读/写操作

- 数据存储中还有个重要问题，就是数据压缩。
- 内存表的空间毕竟是很有限的，当其容量达到一个阈值时，旧的内存表就会被停止使用，并压缩成SSTable格式的文件。

Bigtable的数据压缩形式：

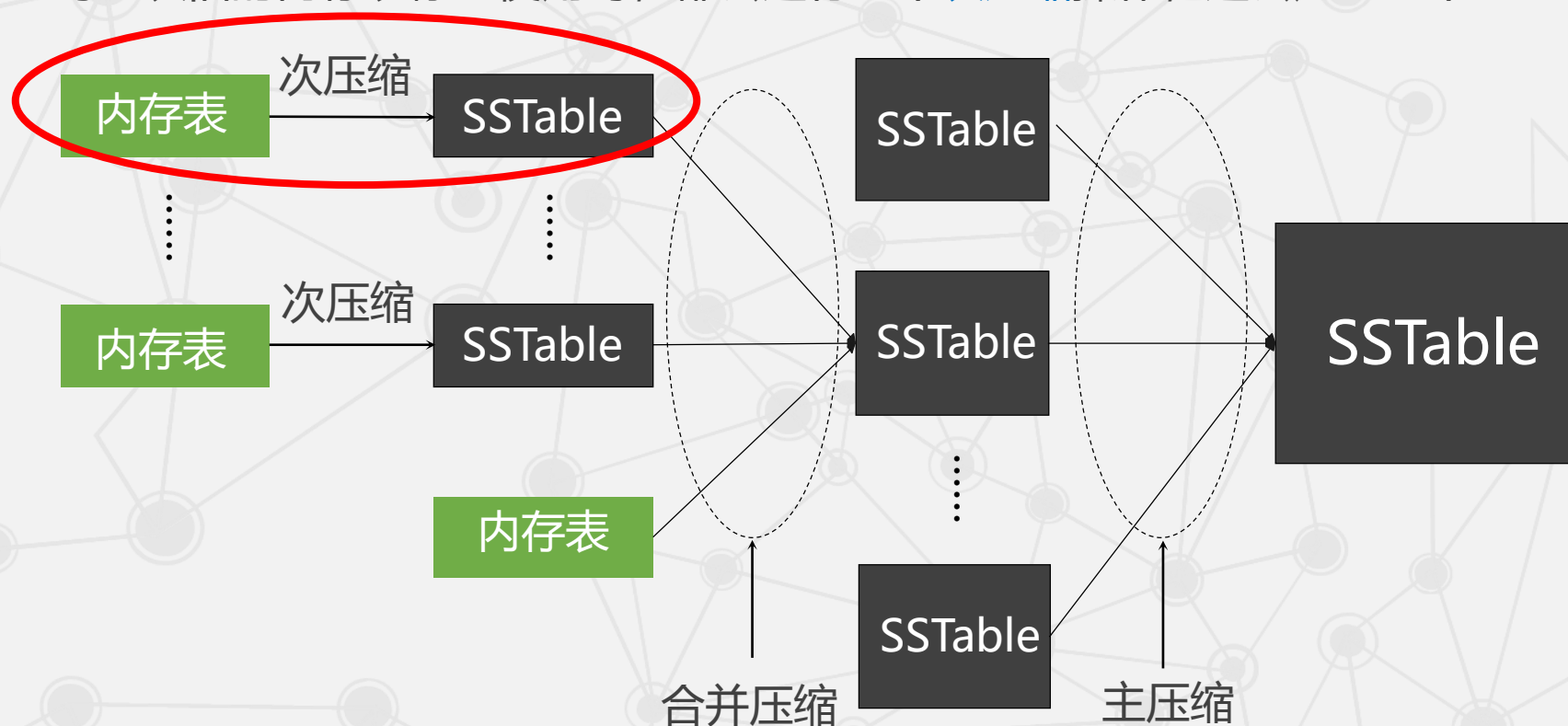
- 次压缩
- 合并压缩
- 主压缩



## 2.4 分布式结构化数据表Bigtable

### ● 3. 子表数据存储及读/写操作

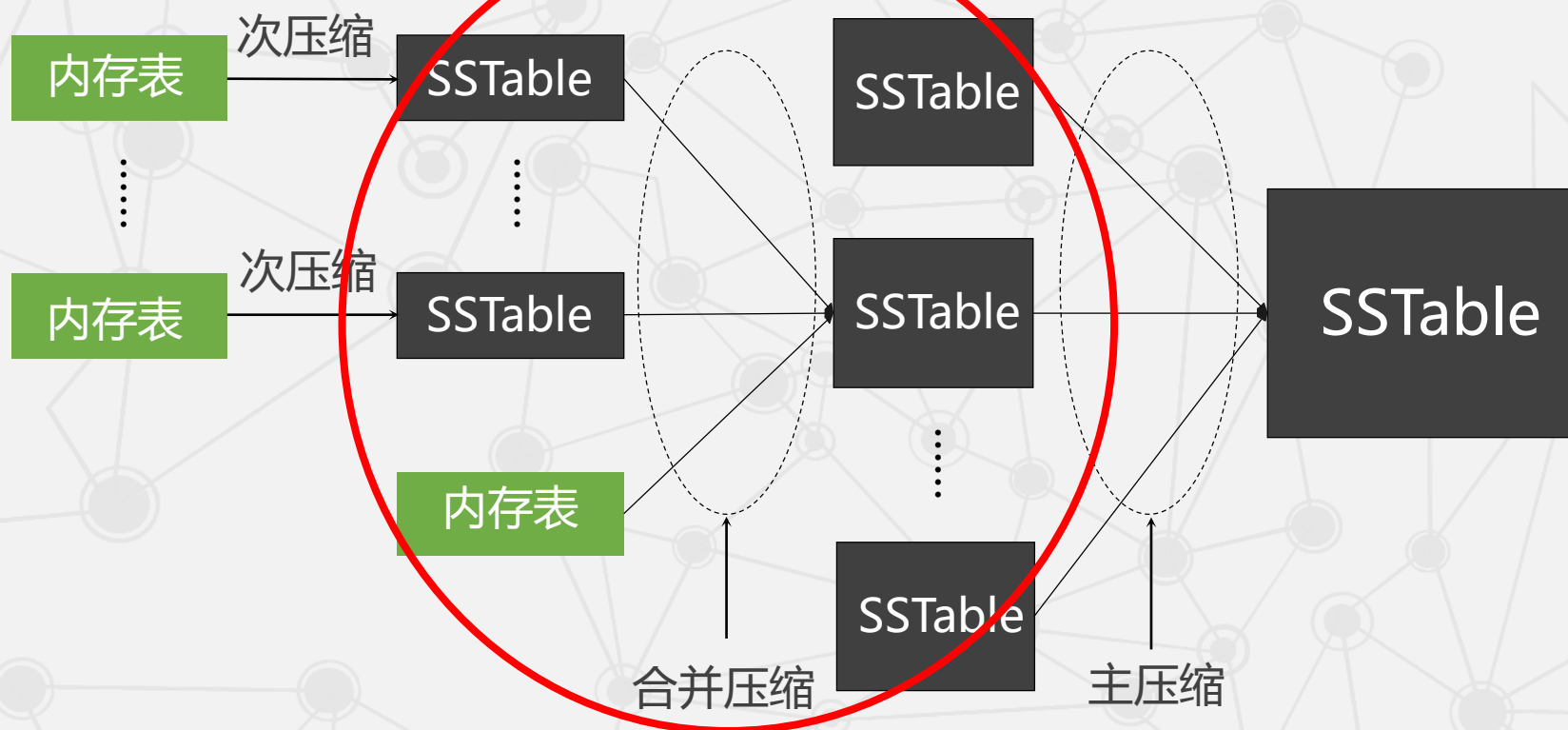
每一次旧的内存表停止使用时，都会进行一个次压缩操作，这会产生一个SSTable。



## 2.4 分布式结构化数据表Bigtable

### ● 3. 子表数据存储及读/写操作

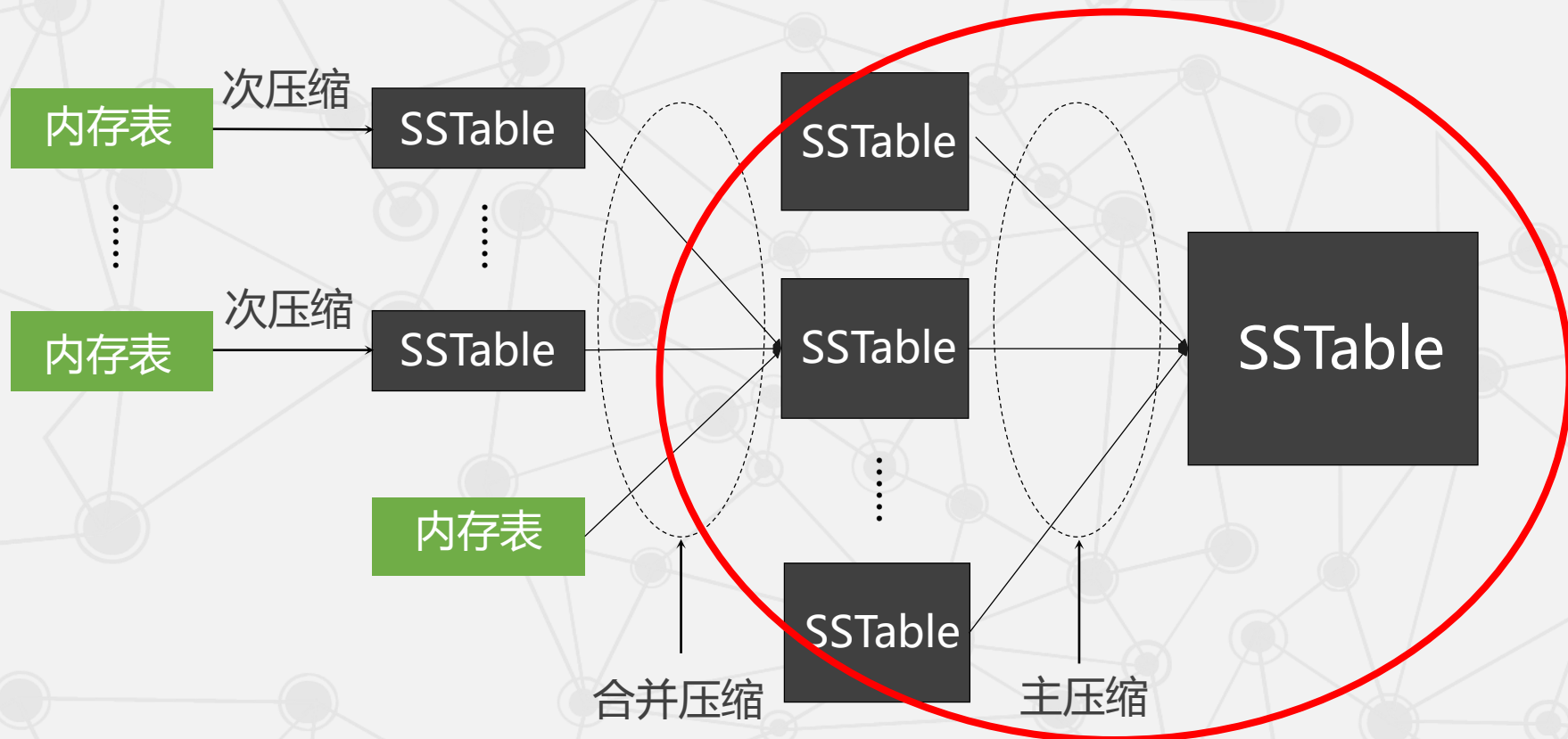
每一次旧的内存表停止使用时，都会进行一个**次压缩**操作，这会产生一个SSTable。



Bigtable会定期执行一次**合并压缩**，将已有的**SSTable**和现有的**内存表**一并进行一次压缩。

## 2.4 分布式结构化数据表Bigtable

### 3. 子表数据存储及读/写操作



**主压缩**是合并压缩的一种，将所有的SSTable一次性压缩成一个大的SSTable文件。主压缩也是**定期**执行，并保证将所有的被压缩数据彻底**删除**。这样，既回收了空间，又保护了敏感数据。

## 2.4 分布式结构化数据表Bigtable

2.4.1 设计动机与目标

2.4.2 数据模型

2.4.3 系统架构

2.4.4 主服务器

2.4.5 子表服务器

► 2.4.6 性能优化

## 2.4 分布式结构化数据表Bigtable

- 上述各种操作已经可以实现Bigtable的所有功能了，但是这些基本的功能很多时候并不是很符合用户的使用习惯，或者执行效率较低。
- 有些功能Bigtable自身已经进行了优化，包括使用缓存、共享式的提交日志，以及利用系统的不变性。
- 除此之外，Bigtable还允许用户个人在基础操作之上，对系统进行一些优化。

这一部分主要介绍3个重要的优化措施：

### ➤ 局部性群组、压缩、布隆过滤器

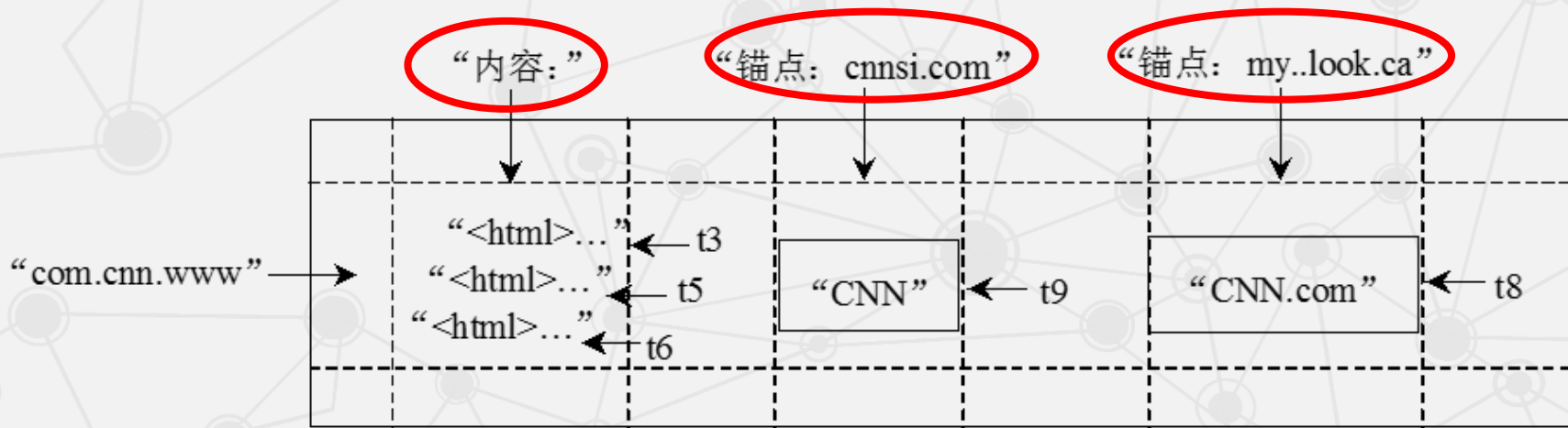
实际上这些技术手段都是一些已有的数据库方法，只不过Google将它具体地应用于Bigtable之中了。



## 2.4 分布式结构化数据表Bigtable

### ● 1. 局部性群组 (Locality groups)

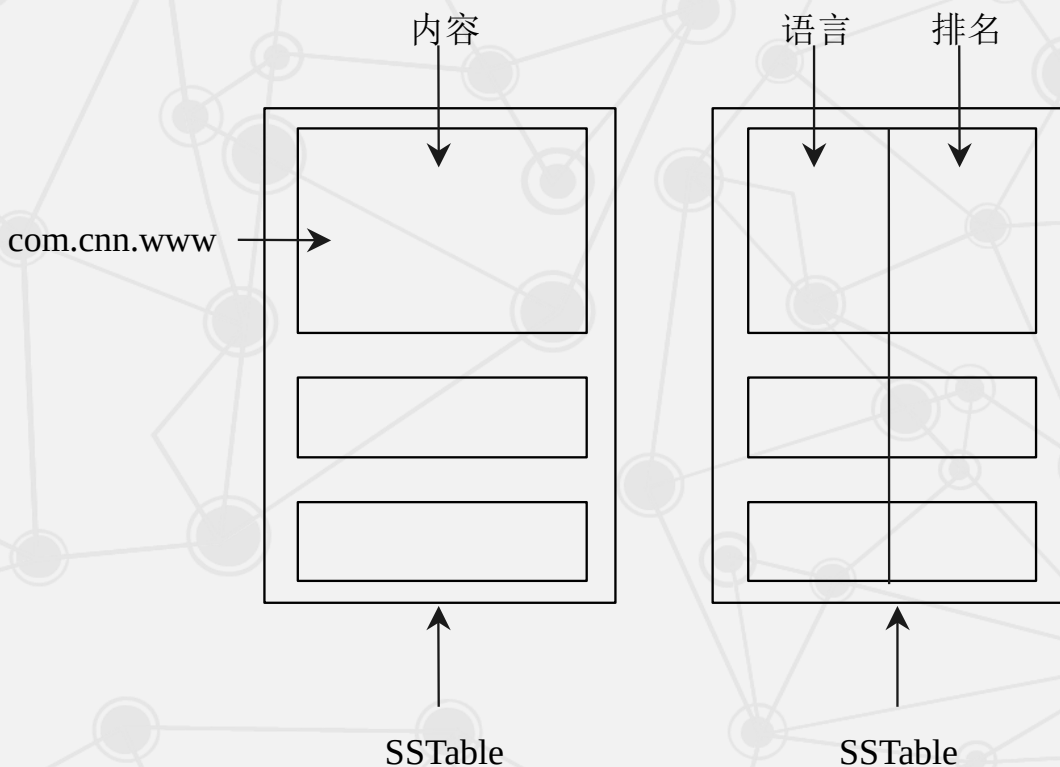
- Bigtable允许用户将原本并不存储在一起的数据以**列族**为单位，根据需要组织在一个单独的SSTable中，以构成一个**局部性群组**。
- 这实际上就是数据库中**垂直分区**技术的一个应用。
- 在被Bigtable保存的网页**列关键字**中，有的用户可能只对网页内容感兴趣，那么它可以通过设置**局部性群组**只看内容这一列。有的则会对诸如网页语言、网站排名等可以用于**分析**的信息比较感兴趣，他也可以将这些设置到一个**群组**中。



## 2.4 分布式结构化数据表Bigtable

### • 1. 局部性群组 (Locality groups)

局部性群组如下图所示。



通过设置局部性群组，用户可以只看自己感兴趣的内容，对某个用户来说的大量**无用信息**无须读取。

对于一些较**小**的且会被经常读取的**局部性群组**，用户可以将其SSTable文件直接加载进**内存**，这可以明显地**改善读取效率**。

## 2.4 分布式结构化数据表Bigtable

### • 2. 压缩

- 压缩可以有效地节省空间，Bigtable中的压缩被应用于很多场合。
- 首先，压缩可以被用在构成局部性群组的SSTable中，可以选择是否对个人的局部性群组的SSTable进行压缩。
- Bigtable中这种压缩是对每个局部性群组独立进行的，虽然会浪费一些空间，但在需要读时，解压速度非常快。
- 通常情况下，用户可以采用两步压缩的方式：

1

利用Bentley & McIlroy方式（BMDiff）在大的扫描窗口将常见的长串进行压缩。

2

采取Zippy技术进行快速压缩，它在一个16KB大小的扫描窗口内寻找重复数据，这个过程非常快。

## 2.4 分布式结构化数据表Bigtable

### • 2. 压缩

- 压缩技术还可以提高子表的恢复速度,
- 当某个子表服务器停止使用后, 需要将所有的子表转移到另一个子表服务器来恢复服务。
- 在转移之前, 要进行两次压缩:

1

第一次压缩减少了提交日志中的未压缩状态, 从而减少了恢复时间。

2

第二次压缩是将第一次压缩后遗留的未压缩空间进行压缩。

- 完成这两步之后, 压缩的文件就会被转移至另一个子表服务器。

## 2.4 分布式结构化数据表Bigtable

### • 3. 布隆过滤器 (Bloom Filter)

- Bigtable向用户提供了一种称为**布隆过滤器**的数学工具。
- 布隆过滤器是巴顿·布隆在**1970**年提出的，实际上它是一个很长的**二进制向量**和一系列**随机映射函数**，在读操作中确定子表的位置时非常有用。

#### 优点

- 布隆过滤器的速度快，省空间
- 不会将一个**存在**的子表判定为不存在

#### 缺点

- 在某些情况下，它会将**不存在**的子表判断为存在。不过这种情况出现的概率非常小，相比于其优点，这个缺点可以忍受。

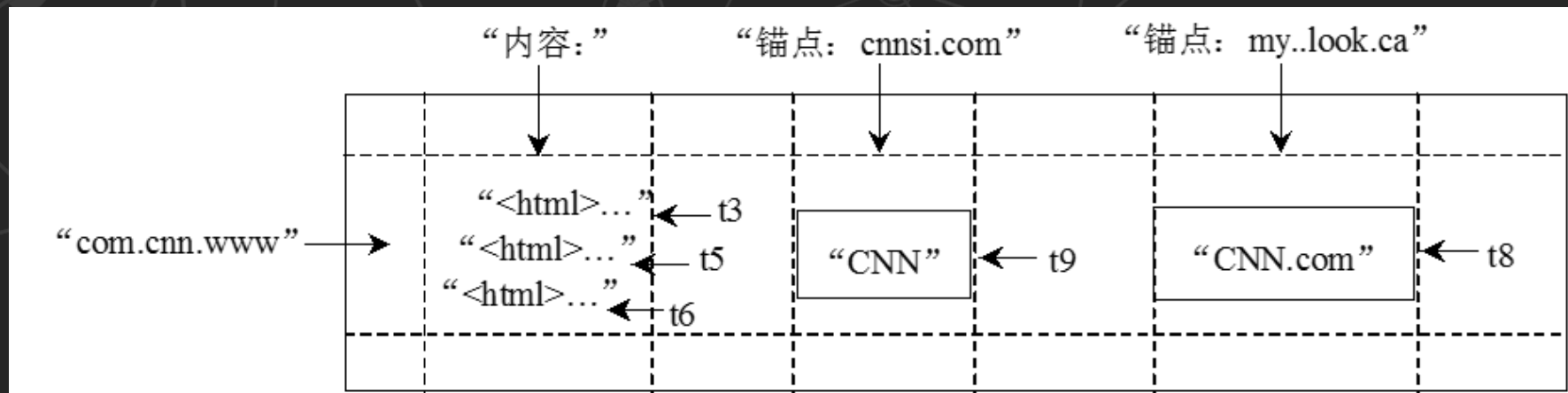
## 2.4 分布式结构化数据表Bigtable

### ● 3. 布隆过滤器 (Bloom Filter)

- 布隆过滤器有许多应用场景，例如：
- 网络爬虫时，判断URL是否已被访问，避免重复抓取。
- 大数据处理时，快速判断数据是否已处理。

# 回顾与思考

1. 两个“锚点”是同一个列族吗？ 是



2. 两个“锚点”的访问控制策略相同吗？

相同。访问控制的单位是列族。

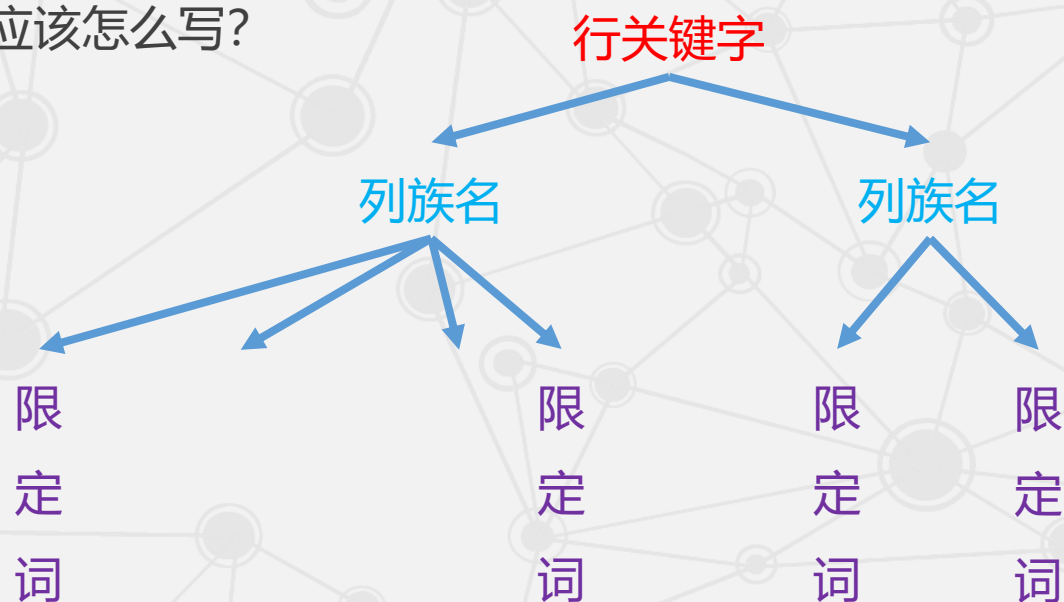
3. Bigtable中负载均衡的单位是什么？

负载均衡的单位是子表。



## 2.4 分布式结构化数据表Bigtable

假设有一个社交媒体平台，用户可以在上面发布帖子，每个帖子都包含一些基本信息，如帖子ID、作者、内容、发布时间、点赞数、评论数等。此外，每个帖子还可能包含一些附加信息，如地理位置、标签、分享链接等。为了高效地存储和访问这些信息，可以使用Bigtable作为存储后端。Bigtable的表应该怎么写？



## 2.4 分布式结构化数据表Bigtable

假设有一个社交媒体平台，用户可以在上面发布帖子，每个帖子都包含一些基本信息，如帖子ID、作者、内容、发布时间、点赞数、评论数等。此外，每个帖子还可能包含一些附加信息，如地理位置、标签、分享链接等。为了高效地存储和访问这些信息，可以使用Bigtable作为存储后端。Bigtable的表应该怎么写？



## 2.4 分布式结构化数据表Bigtable

假设帖子的ID为“post\_123456789”，作者为“张三”，内容为“今天天气真好！”，发布时间为“2024-11-01 12:00:00”，点赞数为100，评论数为20，地理位置为“北京”，标签为“天气、心情”，没有分享链接。那么，在Bigtable中的存储可能如下所示：

| 行键             | 列族              | 列限定符          | 单元值                 |
|----------------|-----------------|---------------|---------------------|
| post_123456789 | basic_info      | author        | 张三                  |
| post_123456789 | basic_info      | content       | 今天天气真好！             |
| post_123456789 | basic_info      | timestamp     | 2023-10-01 12:00:00 |
| post_123456789 | basic_info      | like_count    | 100                 |
| post_123456789 | basic_info      | comment_count | 20                  |
| post_123456789 | additional_info | location      | 北京                  |
| post_123456789 | additional_info | tags          | 天气、心情               |

## 2.4 分布式结构化数据表Bigtable

假设，有一个物联网设备，其ID为“device\_id\_123456789”，该设备配备了温度传感器、湿度传感器和位置传感器。现在，需要存储该设备在一天内产生的数据。

| 行键                  | 列族          | 列限定符        | 时间戳                 | 单元值                          |
|---------------------|-------------|-------------|---------------------|------------------------------|
| device_id_123456789 | device_info | name        | 2024-11-05 08:00:00 | Robot 1                      |
| device_id_123456789 | sensor_data | temperature | 2024-11-05 08:00:00 | 22.5°C                       |
| device_id_123456789 | sensor_data | humidity    | 2024-11-05 08:00:00 | 60%                          |
| device_id_123456789 | sensor_data | location    | 2024-11-05 08:00:00 | 经度: 102.994925、纬度: 29.977977 |
| .....               | .....       | .....       | .....               | .....                        |